

A Theory of Programming

Christian Rinderknecht

Eszterházy Károly Katolikus Egyetem

September 2026

Introduction

This lecture proposes a stepwise introduction to a foundation of programming and the analyses to write reliable and efficient software.

There are many different frameworks. We choose here a formal logic where relations and functions are the main building blocks. This entails that the meaning of **functional programs** can often be made very precise. (We will cover imperative programs later.)

It is a good framework to get a sense of **static analyses**, and, as such, it stands a prominent place in the field of formal methods and **compiler construction**.

String rewriting

Let us consider a string of white and black beads, like $\circ \bullet \bullet \bullet \circ \circ \bullet$, and a one-player game whose unique move consists in replacing two adjacent beads with only one according to the rules

$$\bullet \circ \xrightarrow{\alpha} \bullet \qquad \circ \bullet \xrightarrow{\beta} \bullet \qquad \bullet \bullet \xrightarrow{\gamma} \circ$$

The rules α , β and γ make up a simple **string-rewriting system**.

Rules α and β can be conceived as:

'A black bead absorbs the white bead next to it.'

The goal of this game is to end up with as few beads as possible, so our example may lead to the **rewrites**

$$\circ \bullet \bullet \boxed{\bullet \circ} \circ \bullet \xrightarrow{\alpha} \circ \bullet \bullet \boxed{\bullet \circ} \bullet \xrightarrow{\alpha} \boxed{\circ \bullet} \bullet \bullet \bullet \xrightarrow{\beta} \bullet \bullet \boxed{\bullet \bullet} \xrightarrow{\gamma} \bullet \boxed{\bullet \circ} \xrightarrow{\alpha} \boxed{\bullet \bullet} \xrightarrow{\gamma} \circ$$

where the part of the string to be rewritten next is framed.

Normal forms and termination

Other compositions of the rules lead to the same result $\circ$ as well. Some others bring all-white or all-black strings, like $\circ\circ$ and $\bullet$.

Strings that can not be further rewritten, or **reduced**, are called **normal forms**. We may wonder whether all strings have a normal form, and, if so, if it is unique and, furthermore, if it is either all-white or all-black.

First, let us note that the system is **terminating**, that is, there is no infinite chain of rewrites, because the number of beads strictly decreases in all the rules.

This is not a necessary condition in general, for instance, $\circ\bullet \xrightarrow{\beta} \bullet\circ\circ$ would preserve termination because the composition $\beta\alpha\alpha$ would be equivalent to the original rule β .

In particular, this means that any string has a normal form.

Invariants

Second, notice how the parity of the number of black beads is **invariant** through each rule and how there is no rewrite rule for two adjacent white beads.

Therefore, if there are $2p$ initial black beads, then composing rules α and β lead to an all-black string, which can be reduced by applying rule γ to contiguous pairs of beads into an all-white string made of p beads.

Otherwise, the same all-black string can be reduced by applying alternatively γ and β on the left end or γ and α on the right end, yielding $\circ$.

Similarly, if there is an initial odd number of black beads, we always end up with one black bead.

Confluence

Consequently, normal forms are not unique. A simple example is

$$\circ\circ \xleftarrow{\gamma} \bullet\bullet\circ \xrightarrow{\alpha} \bullet\bullet \xrightarrow{\gamma} \circ$$

Systems where normal forms are unique are **confluent**.

If we think of a rewrite step as an elementary computation, it becomes clear that confluence is a desirable property for carrying out computations, as we would expect, for instance, an arithmetic expression to have the same value no matter the order of the small steps.

A systemic order of rule application is called a **reduction strategy**.

Completing a system

We can obtain confluence by adding the rule δ :

$$\bullet \circ \xrightarrow{\alpha} \bullet \qquad \circ \bullet \xrightarrow{\beta} \bullet \qquad \bullet \bullet \xrightarrow{\gamma} \circ \qquad \circ \circ \xrightarrow{\delta} \circ$$

The result of the game is now always one bead whose colour depends on the original parity of the black beads as before.

To see why, let us consider first that two non-overlapping parts of a string can be rewritten in parallel, so they are not a concern.

The interesting cases occur when two applications of rules (maybe of the same rule) lead to different strings because they do overlap. For instance,

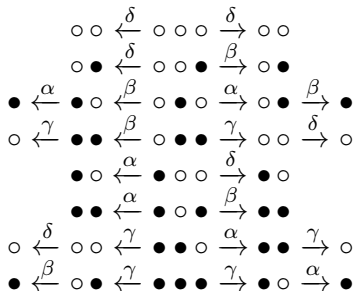
$$\circ \circ \xleftarrow{\gamma} \bullet \bullet \circ \xrightarrow{\alpha} \bullet \bullet$$

The important point is that $\circ \circ$ and $\bullet \bullet$ can be rewritten into $\circ$ at the next step by δ and γ , respectively.

Critical pairs

In general, what matters is that all pairs of strings resulting from the application of overlapping rules, called **critical pairs**, can be rewritten to the same string: they are **joinable**.

In our example, all interactions occur on substrings made of three beads, so we must examine eight cases:



In all the cases, the divergences are joinable in one step at most.

Local confluence

If all critical pairs are joinable after a divergence in one ore more rewrites, the system is **locally confluent**.

Moreover, if the system is also terminating, then **every string has exactly one normal form**.

Sometimes, as seen previously, a non-confluent system can be completed to become confluent, via a **completion procedure**, for example, in the case of terminating systems, the **Knuth-Bendix semi-algorithm**.

Ground and linear rewrite systems

The system we defined is **ground**, that is, it involves no variables.

These allow a finite system to denote an infinite number of ground rules if variables range over infinite sets, or simply, they reduce the size of rewrite systems.

For instance, our continued example is equivalent to

$$\bullet \circ \xrightarrow{\alpha} \bullet \qquad \circ \ x \xrightarrow{\beta+\delta} x \qquad \bullet \bullet \xrightarrow{\gamma} \circ$$

If we accept multiple occurrences of a variable on the left-hand side of a rule (that is, not left-linear), we can further decrease the size of the system:

$$x \ x \xrightarrow{\gamma+\delta} \circ \qquad x \ y \xrightarrow{\alpha+\beta} \bullet$$

Ordered rewrite systems

Notice that there is now an implicit order over the rules: the rule $\gamma + \delta$ must be examined first for a **match** with a part of the current string, because it is included in the second (set $x = y$ in $\alpha + \beta$).

Non-linear systems are equational theories due the implicit equality on substructures, which is complex: we will limit ourselves to **linear systems**.

In general, the order in which the rules are written on the page is their **logical order**.

Moreover, let us remark that the system does not specify that x must either be $(\circ)$ or $(\bullet)$: this should be stated nevertheless somewhere else.

Generally speaking, this means that the **type** of the variables has to be defined elsewhere or inferred from the variable uses.

A simple calculator

Perhaps, after playing with beads, the simplest thing is an integer calculator, that is, integers and the four basic arithmetic operators.

An **expression** may be built using these operations, integers and parentheses. For example $(2 + 3) \times (4 + 5)$ is an expression.

A **value** is an integer, therefore it is a special case of expression.

What a calculator does is to match an input expression with a value, the result. For example, the above expression evaluates in 45.

This process is called **evaluation**.

Evaluation as rewriting expressions / Reductions

The evaluation of arithmetic expressions can be defined as a series of transformations leading to a value (the result). These transformations are called **rewrites** and a series of rewrites is a **reduction** or, more generally, a **computation**.

As we learnt in school long time ago, the way to perform these rewrites is to select a part of the current expression consisting of an operator and its arguments *as values*, then to replace this sub-expression by the operation result.

In the following reduction, let us underline the sub-expression to be rewritten and print in bold the result of each step:

$$(2 + 3) \times (\underline{4 + 5}) \rightarrow (\underline{2 + 3}) \times \mathbf{9} \rightarrow \mathbf{5} \times \underline{9} \rightarrow \mathbf{45}$$

Strategy of evaluation and rewrite systems

Note that there are different ways to choose the sub-expression. For example:

$$(\underline{2 + 3}) \times (4 + 5) \rightarrow \mathbf{5} \times (\underline{4 + 5}) \rightarrow \underline{5 \times 9} \rightarrow \mathbf{45}$$

It is usual to write, as a summary: $(2 + 3) \times (4 + 5) \xrightarrow{*} 45$ or $(2 + 3) \times (4 + 5) = 45$.

So, there are two concepts involved here:

1. the selection of a sub-expression made of an operator and its arguments as values,
2. the rewriting of such sub-expression into a value.

They are defined by a **rewrite system**.

Rewrite rules

A rewrite system is a set of **rewrite rules**.

Rewriting rules specify how to perform a rewrite.

For example, let us assume that we have an infinite set of rewriting rules, that is. an infinite rewrite system, like:

$$0 + 0 \rightarrow 0$$

$$0 + 1 \rightarrow 1$$

$$\dots \rightarrow \dots$$

$$124 - 57 \rightarrow 67$$

$$\dots \rightarrow \dots$$

Conditional rewrite rules and meta-variables

This rewrite system is not enough for the kind of expressions we want to evaluate, where it is possible to combine operations, like $(2 + 3) \times (4 + 5)$.

So let us extend our system with rules that specify the sub-expressions we can rewrite.

Consider the multiplication:

$$e_1 \times e_2 \rightarrow e'_1 \times e_2 \quad \text{if } e_1 \rightarrow e'_1$$

where e_1 , e'_1 and e_2 denote any expression and are called **meta-variables**.

Instance of a rewrite rule

An **instance** of this rule is obtained when the meta-variables are replaced by expressions.

For example, $(2 + 3) \times (4 + 5) \rightarrow 5 \times (4 + 5)$ is an instance since $2 + 3 \rightarrow 5$. To understand why, replace e_1 by $2 + 3$, e'_1 by 5 and e_2 by $4 + 5$.

Let us rewrite $((2 + 3) \times 6) \times (4 + 5)$. First, we see that the previous rule has the required shape, i.e., the left side of the arrow is $e_1 \times e_2$ and if $e_1 = (2 + 3) \times 6$ and $e_2 = 4 + 5$ then $e_1 \times e_2$ is exactly the expression we wish to rewrite. So, we replace e_1 and e_2 by their associated expressions in the rule and we get an instance of it:

$$((2 + 3) \times 6) \times (4 + 5) \rightarrow e'_1 \times (4 + 5) \quad \text{if } (2 + 3) \times 6 \rightarrow e'_1$$

Therefore, we now know that we have to rewrite $(2 + 3) \times 6$.

Instance of a rewrite rule

We need another instance of the same rule. This time, let $e_1 = 2 + 3$ and $e_2 = 6$. We get

$$(2 + 3) \times 6 \rightarrow e'_1 \times 6 \quad \text{if } 2 + 3 \rightarrow e'_1$$

It is important to understand that this latter e'_1 is not the same as the former: they belong to two different instances of the same rule. The latter e'_1 is actually easy to find, since we have all the rules for basic cases as $2 + 3 \rightarrow 5$. Hence, $e'_1 = 5$ and the second instance is now complete:

$$(2 + 3) \times 6 \rightarrow 5 \times 6 \quad \text{if } 2 + 3 \rightarrow 5$$

Going backwards, this implies that the former e'_1 is 5×6 . So, the first instance is

$$((2 + 3) \times 6) \times (4 + 5) \rightarrow (5 \times 6) \times (4 + 5) \quad \text{if } 2 + 3 \rightarrow 5$$

Instance of a rewrite rule

Our rule for multiplication, $e_1 \times e_2 \rightarrow e'_1 \times e_2$, actually depends on another, $e_1 \rightarrow e'_1$. It is usual to write them vertically:

$$\frac{e_1 \rightarrow e'_1}{e_1 \times e_2 \rightarrow e'_1 \times e_2} \text{MULT}_1$$

This is called an **inference rule**. Note that it has a name, MULT_1 . The rewrite rule on the top, acting as a condition for the rule on the bottom, is called a **premise**. The rule at the bottom is called a **conclusion**.

A **substitution** is a mapping from meta-variables to expressions which applies to an inference rule or rewrite rule. For example, let σ be the following substitution: $\{e_1 \mapsto (2 + 3) \times 6; e_2 \mapsto 4 + 5\}$. This notation is equivalent to $\sigma(e_1) = (2 + 3) \times 6$ and $\sigma(e_2) = 4 + 5$.

Instance of a rewrite rule

Now we can make a clear definition of the notion of ‘instance of a rule’:

An instance of a rule is a rule in which some meta-variables e_i are replaced by expressions $\sigma(e_i)$.

For example, here is the instance of MULT_1 using previously defined substitution σ , and noted $\sigma(\text{MULT}_1)$:

$$\frac{(2 + 3) \times 6 \rightarrow e'_1}{((2 + 3) \times 6) \times (4 + 5) \rightarrow e'_1 \times (4 + 5)} \sigma(\text{MULT}_1)$$

Instance of a rewrite rule

Now how do we find (a mapping for) e'_1 ?

We define another substitution $\sigma' = \{e_1 \mapsto 2 + 3; e_2 \mapsto 6\}$ and apply it:

$$\frac{2 + 3 \rightarrow e'_1}{(2 + 3) \times 6 \rightarrow e'_1 \times 6} \sigma'(\text{MULT}_1)$$

Note that the meta-variable e'_1 in $\sigma'(\text{MULT}_1)$ is not the same as in $\sigma(\text{MULT}_1)$: they should have a different mapping.

We have a basic rewrite rule $2 + 3 \rightarrow 5$, thus we extend σ' with $e'_1 \mapsto 5$ and get

$$\frac{2 + 3 \rightarrow 5}{(2 + 3) \times 6 \rightarrow 5 \times 6} \sigma'(\text{MULT}_1)$$

Instance of a rewrite rule

Therefore we can extend σ with $e'_1 \mapsto 5 \times 6$ and we get

$$\frac{(2 + 3) \times 6 \rightarrow 5 \times 6}{((2 + 3) \times 6) \times (4 + 5) \rightarrow (5 \times 6) \times (4 + 5)} \sigma(\text{MULT}_1)$$

We can summarise this rewrite using the two instances of MULT_1 as

$$\frac{\frac{2 + 3 \rightarrow 5}{(2 + 3) \times 6 \rightarrow 5 \times 6} \sigma'(\text{MULT}_1)}{((2 + 3) \times 6) \times (4 + 5) \rightarrow (5 \times 6) \times (4 + 5)} \sigma(\text{MULT}_1)$$

Instance of a rewrite rule

But what about the rewrite of an expression like $7 \times (4 + 5)$?

Let us define another substitution on MULT_1 :

$$\sigma' = \{e_1 \mapsto 7; e_2 \mapsto 4 + 5\}.$$

The corresponding instance is

$$\frac{7 \rightarrow e'_1}{7 \times (4 + 5) \rightarrow e'_1 \times (4 + 5)} \sigma'(\text{MULT}_1)$$

But there is no rule of shape ' $v \rightarrow \dots$ ', where v is a *value* (because a value is, by definition, an expression completely reduced).

Completing the multiplication rule

This means that we need another rule for the evaluation of the *right* argument of $(\times)$:

$$\frac{e_2 \rightarrow e'_2}{e_1 \times e_2 \rightarrow e_1 \times e'_2} \text{MULT}_2$$

This inference rule says that we can rewrite a product by rewriting its right argument.

For example, here is the instance $\sigma'(\text{MULT}_2)$:

$$\frac{4 + 5 \rightarrow 9}{7 \times (4 + 5) \rightarrow 7 \times 9} \sigma'(\text{MULT}_2)$$

Strategy

But now, something interesting happens.

Consider again $(2 + 3) \times (4 + 5)$: both MULT_1 and MULT_2 can be instantiated by substitution $\sigma'' = \{e_1 \mapsto 2 + 3; e_2 \mapsto 4 + 5\}$:

$$\frac{2 + 3 \rightarrow 5}{(2 + 3) \times (4 + 5) \rightarrow 5 \times (4 + 5)} \sigma''(\text{MULT}_1)$$

$$\frac{4 + 5 \rightarrow 9}{(2 + 3) \times (4 + 5) \rightarrow (2 + 3) \times 9} \sigma''(\text{MULT}_2)$$

Strategy

This means that *our system does not specify the order of evaluation of the arguments to $(\times)$* .

In other words, our rewrite system does not select only one sub-expression to rewrite at each step.

We need a **strategy** to complete it.

Finiteness and soundness of reductions

At this point, it is clear that these reductions satisfy the following properties:

- **Finiteness.** All the reductions are finite, i.e., all the computations terminate.
- **Soundness.** All the reductions of an expression end with the same value, if any. (Think confluence.)

About the second point, note indeed that *some expressions have no value*, as

$$(\underline{2 + 3}) / (4 - 4) \rightarrow \mathbf{5} / (\underline{4 - 4}) \rightarrow 5 / \mathbf{0} \nrightarrow$$

The reduction is finite but does not end with a value: this situation corresponds to an **error**, usually called **run-time error** in programming languages. Expression $5/0$ is **stuck**.

Another model of run-time errors

We can modify the calculator so that, in case of error (here, the only one is the division by zero), the result of a rewrite is a special value, called NaN (*Not a Number*). Then we add the following rules:

$$e/0 \rightarrow \text{NaN} \quad \text{DIVZERO} \qquad \text{NaN}/e \rightarrow \text{NaN} \quad \text{DIV-ERR}_1$$

$$e/\text{NaN} \rightarrow \text{NaN} \quad \text{DIV-ERR}_2 \qquad \text{NaN} \times e \rightarrow \text{NaN} \quad \text{MULT-ERR}_1$$

$$e \times \text{NaN} \rightarrow \text{NaN} \quad \text{MULT-ERR}_2 \qquad \text{NaN} + e \rightarrow \text{NaN} \quad \text{ADD-ERR}_1$$

$$e + \text{NaN} \rightarrow \text{NaN} \quad \text{ADD-ERR}_2 \qquad \text{NaN} - e \rightarrow \text{NaN} \quad \text{SUB-ERR}_1$$

$$e - \text{NaN} \rightarrow \text{NaN} \quad \text{SUB-ERR}_2$$

Another model of run-time errors

The rationale of this system is that once an error occurs, no other computations (leading to integer values) are required and the reduction can end as soon as possible.

In this case:

$$(2 + 3)/(4 - 4) \rightarrow 5/(4 - 4) \rightarrow 5/0 \rightarrow \text{NaN}$$

or, even the shortest possible:

$$(2 + 3)/(4 - 4) \rightarrow (2 + 3)/0 \rightarrow \text{NaN}$$

Rephrasing soundness

The interesting phenomenon here is that, now, any reduction ends in a value (since errors are values). In the previous rewrite system, any reduction ends either in a value or a **stuck expression**, i.e., an expression that cannot be rewritten further.

With the new system, that explicitly specifies the reduction of the NaN error, the soundness property can be rephrased as

All the reductions of an expression end with the same value.

Obviously, the drawback of the new system is that it requires a lot of additional rules, and, as the system is augmented with new rules, new rules for the errors have to be added... or are forgotten by mistake.

Specifying the strategy

It is possible to force an order of evaluation on the arguments of $(\times)$. Assume we want to reduce the left argument first.

The idea is to change MULT_2 so that instead of e_1 , which stands for any expression, we specify a value v :

$$\frac{e_2 \rightarrow e'_2}{v \times e_2 \rightarrow v \times e'_2} \text{NEW-MULT}_2$$

This way, it is not possible anymore to instantiate this rule with expression $(2 + 3) \times (4 + 5)$ because $2 + 3$ is not a value.

Specifying the strategy

The only possible reduction is:

$$\frac{2 + 3 \rightarrow 5}{(2 + 3) \times (4 + 5) \rightarrow 5 \times (4 + 5)} \sigma_1(\text{MULT}_1)$$

$$\frac{4 + 5 \rightarrow 9}{5 \times (4 + 5) \rightarrow 5 \times 9} \sigma_2(\text{NEW-MULT}_2)$$

where $\sigma_1 = \{e_1 \mapsto 2 + 3; e_2 \mapsto 4 + 5\}$ and $\sigma_2 = \{v \mapsto 5; e_2 \mapsto 4 + 5\}$.

Determinism

We have specified the strategy **in** the rewrite system.

Also, with `NEW-MULT2`, given an expression, there is only one rule that can be applied (or none): this is sometimes called **determinism**.

Determinism. If $e_1 \rightarrow e'_1$ and $e_2 \rightarrow e'_2$ then $e'_1 = e'_2$.

This is a stronger property than soundness, i.e., it implies soundness. Indeed, determinism implies that there is only one reduction from an expression to its value (if any).

Pragmatics of programming languages

In many programming languages, the order of evaluation of operator arguments is not specified, which usually means that there is a sentence in the official document which defines the language like: 'The order of evaluation of the arguments of operators and functions is not specified.'

Formally, this means that the underlying rewrite system is **non-deterministic**.

One notable exception is Java, which specifies that the arguments are always evaluated from left to right, i.e., following the order of writing.

Of course, if an order is specified, as in Java, the compilers of these languages must implement it. Otherwise one is chosen arbitrarily and may change from one release to another.

Rewrite systems as models of languages

By studying rewrite systems, we can learn the basic concepts involved in programming languages, because rewrite systems are extremely simple.

There are some languages, like Maude, whose programs actually are rewrite systems.

Some compilers transform the input program (called **source**) and output a program, called **byte-code**, which can be reduced by a rewrite system.

Rewrite systems as models of languages

This is the case of Java. This is why, in general, we need a **Java virtual machine** (JVM) to run the program: this JVM performs the reductions according to a rewrite system.

Even if there is no byte-code, we always can interpret what the program does by studying a rewrite system which models it.

Sequentiality and parallelism

Some programming languages, like Ada, allow the programmer to specify that different parts of its program can actually be executed separately: this is called, in general, **parallelism**. This parallelism is possible only if the underlying rewrite system is non-deterministic.

Indeed, if all the strategies lead to the same value (soundness property), it is possible to reduce different parts of an expression in parallel if the rewrite system allows it.

Sequentiality and parallelism

For example, the first argument of $(\times)$ can be computed *at the same time* as its second argument. In the case of $(2 + 3) \times (4 + 5)$, this means that we can reduce in parallel $2 + 3$, with MULT_1 , and $4 + 5$, with MULT_2 , then 5×9 is finally rewritten in 45 (after **synchronisation** of the two reductions).

Order of evaluation in C

In case of C, the order of evaluation arguments of arithmetic operators and functions is not specified, it actually depends on the compiler and its version. Therefore it would be dangerous to rely on the order of evaluation in C.

Why is the order left unspecified? Simply because of possible **side-effects**. Imagine the following situation:

```
int n = 0;
int f () { n++; return n; };
int g () { n = 2; return n; };
int main () { return f() + g() + n; }
```

What is the result?

Boolean expressions

Let $\wedge$ represent the conjunction ('and'), $\vee$ the disjunction ('or'), $\neg$ the negation ('not'), and let the two values true and false.

The definition of the conjunction of values is very simple:

$$\text{true} \wedge \text{true} \rightarrow \text{true}$$

$$\text{true} \wedge \text{false} \rightarrow \text{false}$$

$$\text{false} \wedge \text{true} \rightarrow \text{false}$$

$$\text{false} \wedge \text{false} \rightarrow \text{false}$$

Boolean expressions made simpler

This system can even be made simpler, by means of meta-variables:

$$\text{true} \wedge \text{true} \rightarrow \text{true} \quad \text{TRUE}$$

$$\text{false} \wedge v \rightarrow \text{false} \quad \text{FALSE}_1$$

$$v \wedge \text{false} \rightarrow \text{false} \quad \text{FALSE}_2$$

where v denotes any value, i.e., either true or false.

Boolean expressions

So, if we define the rewrite of boolean expressions (not just the values true and false) as

$$\frac{e_1 \rightarrow e'_1}{e_1 \wedge e_2 \rightarrow e'_1 \wedge e_2} \wedge_1$$

$$\frac{e_2 \rightarrow e'_2}{e_1 \wedge e_2 \rightarrow e_1 \wedge e'_2} \wedge_2$$

then, for example,

$$\frac{\text{false} \wedge \text{true} \rightarrow \text{false} \quad \text{FALSE}_1}{(\text{false} \wedge \text{true}) \wedge (\text{true} \wedge \text{true}) \rightarrow \text{false} \wedge (\text{true} \wedge \text{true})} \sigma_1(\wedge_1)$$

where $\sigma_1 = \{e_1 \mapsto \text{false} \wedge \text{true}; e_2 \mapsto \text{true} \wedge \text{true}; e'_1 \mapsto \text{false}\}$.

Boolean expressions

Then

$$\frac{\text{true} \wedge \text{true} \rightarrow \text{true} \quad \text{TRUE}}{\text{false} \wedge (\text{true} \wedge \text{true}) \rightarrow \text{false} \wedge \text{true}} \sigma_2(\wedge_2)$$

where $\sigma_2 = \{e_1 \mapsto \text{false}; e_2 \mapsto \text{true} \wedge \text{true}; e'_1 \mapsto \text{true}\}$.

and, finally, we can instantiate FALSE_1 with $\sigma_3 = \{v \mapsto \text{true}\}$:

$$\text{false} \wedge \text{true} \rightarrow \text{false} \quad \sigma_3(\text{FALSE}_1)$$

In this example, we computed both arguments of $\wedge$ to reach the value.

Boolean expressions

But the inspection of the basic rules shows that if one of the arguments of $\wedge$ reduces to false, then there is no need to reduce the other argument: the rewrite will always be false. Therefore, it is more efficient to extend the rules on values to the expressions:

$$\text{false} \wedge e \rightarrow \text{false} \quad \text{FALSE}_1 \qquad e \wedge \text{false} \rightarrow \text{false} \quad \text{FALSE}_2$$

$$\text{true} \wedge e \rightarrow e \quad \text{TRUE}_1 \qquad e \wedge \text{true} \rightarrow e \quad \text{TRUE}_2$$

where e represents any boolean expression. Then, now

$$\text{false} \wedge (\text{true} \wedge \text{true}) \rightarrow \text{false} \quad \sigma_3(\text{FALSE}_1)$$

where $\sigma_3 = \{e \mapsto \text{true} \wedge \text{true}\}$.

Boolean expressions

Try to reduce

$$(\text{true} \wedge (\text{false} \wedge \text{true})) \wedge (\text{true} \wedge (\text{true} \wedge \text{true}))$$

Note that this rewrite system is not deterministic. If we enforce that the left argument of $\wedge$ is completely reduced first, then we can check if it is true or false: in the latter case, we do not need to reduce the right argument.

Boolean expressions

This is easy to specify:

$$\text{true} \wedge e \rightarrow e \quad \wedge_{\text{TRUE}} \qquad \text{false} \wedge e \rightarrow \text{false} \quad \wedge_{\text{FALSE}}$$

$$\frac{e_1 \rightarrow e'_1}{e_1 \wedge e_2 \rightarrow e'_1 \wedge e_2} \wedge$$

Many programming languages, as C, use this strategy. It allows the programmers to write handy code like

```
while (index < max && tab[index] > 0) ...
```

Boolean expressions

Note that this strategy (left argument before the right one) is not optimal in general: the first argument may reduce to true. *Finding an optimal strategy is not always possible.* For example, we could add another rule that shortens sometimes the reductions:

$$e \wedge e \rightarrow e \wedge_{\text{REFL}}$$

This rule explicitly specifies the **reflexivity** of the conjunction. But the system becomes non-deterministic because there is now an overlap between rule $\wedge_{\text{REFL}}$ and both $\wedge_{\text{TRUE}}$ and $\wedge_{\text{FALSE}}$ (consider for example $\text{true} \wedge \text{false}$).

Determinism versus non-determinism

Let us summarise the concepts of determinism and non-determinism.

- Determinism means that, for any expression, there is only one rule to rewrite it. This implies that, every time a program is run with the same input, the output will be the same. This is a useful property, called soundness, in particular for debugging.
- Non-determinism means that, for any expression, there may be several rules to rewrite it. Moreover,
 - if the rewrite system is sound, the output will always be the same for any run on the same input, even if the trace may be different each time, which makes the debugging more difficult, but enable parallel reductions.
 - the rewrite system may be unsound.

Determinism versus non-determinism

Some programming languages have features that may be both non-deterministic and unsound.

Perhaps the most infamous case is in C, where variables do not need to be initialised at their declaration:

```
int a;
```

If the programmer forgets to give a value to the variable, any access to it is unspecified.

Practically, this means that whatever is located in the memory at the location of the variable will be used at each read access. Of course, this value may change from one run of the program to another, even on the same input.

Commutativity and its consequences

We may try another rewrite system for $\wedge$:

$$\text{true} \wedge e \rightarrow e \quad \text{TRUE}$$

$$\text{false} \wedge e \rightarrow e \quad \text{FALSE}$$

$$\frac{e_1 \rightarrow e'_1}{e_1 \wedge e_2 \rightarrow e'_1 \wedge e_2} \wedge$$

$$e_1 \wedge e_2 \rightarrow e_2 \wedge e_1 \quad \text{COMM}$$

While this system is in theory fine, there are two practical problems:

1. it is **not deterministic** due to COMM that overlaps with all the other rules;
2. it may **not terminate** due to rule COMM:

$$\begin{aligned} \text{true} \wedge \text{false} &\xrightarrow{\text{COMM}} \text{false} \wedge \text{true} \xrightarrow{\text{COMM}} \text{true} \wedge \text{false} \\ &\xrightarrow{\text{COMM}} \text{false} \wedge \text{true} \rightarrow \dots \end{aligned}$$

Non-termination

Now some reductions may not be finite, which models **non-termination**.

This particular kind of non-termination, due to commutativity, is the model in programming languages of so-called '**infinite loops**' that do not allocate memory, i.e., that do not require additional memory at each loop step. Equivalently, it corresponds to a *recursive call* like

```
bool and(bool e1, bool e2) {  
    ... return and(e2, e1); ... }
```

Anyway, in the presence of infinite reductions, the soundness property must be weakened as follows:

Weak soundness. *All finite reductions of an expression end with the same value, if any.*

Non-termination

Let us define the negation:

$$\neg \text{true} \rightarrow \text{false}$$

$$\neg \text{false} \rightarrow \text{true}$$

It is also well known that the negation satisfies $e = \neg\neg e$. So we could imagine another rewrite rule

$$\neg\neg e \rightarrow e$$

This is fine, this rule simplifies double negations. But what if we had the converse rule?

$$e \rightarrow \neg\neg e \quad \text{where } e \text{ is not a value.}$$

This allows infinite reductions: $e \rightarrow \neg\neg e \rightarrow e \rightarrow \neg\neg e \rightarrow \dots$

Summary

There are three kinds of reductions:

1. Finite and ending in a value

$$e_1 \rightarrow e_2 \rightarrow \cdots \rightarrow v$$

2. Finite but ending in a stuck expression

$$e_1 \rightarrow e_2 \rightarrow \cdots \rightarrow e_n \nrightarrow$$

3. Infinite

$$e_1 \rightarrow e_2 \rightarrow \cdots$$

Conditional expressions

At this point, boolean expressions can be

- **true** (a value)
- **false** (a value)
- $e_1 \wedge e_2$
- $e_1 \vee e_2$
- $\neg e_1$

where e_1 and e_2 are boolean expressions. Let us extend our boolean expressions with a **conditional expression**:

- **if** e_1 **then** e_2 **else** e_3

Conditional expressions

It is easy to extend the rewrite system to cope with conditionals:

if true then e_2 **else** $e_3 \rightarrow e_2$ IF-TRUE

if false then e_2 **else** $e_3 \rightarrow e_3$ IF-FALSE

$$\frac{e_1 \rightarrow e'_1}{\text{if } e_1 \text{ then } e_2 \text{ else } e_3 \rightarrow \text{if } e'_1 \text{ then } e_2 \text{ else } e_3} \text{ IF}$$

This means that the boolean expression e_1 must be completely reduced *before* the others. This is the usual way in programming languages, the rationale being to avoid unnecessary computations (evaluating e_2 and e_3 is not necessary).

Term rewriting

More general are the **term-rewriting systems**, where a **term** is a mathematical object built on tuples, integers and variables.

Let us consider the following totally ordered system:

$$\begin{aligned}(0, m) &\xrightarrow{1} m; \\ (n, m) &\xrightarrow{2} (n - 1, n \times m); \\ n &\xrightarrow{3} (n, 1).\end{aligned}$$

Arithmetic operators $(-)$ and $(\cdot)$ are defined out of the system and m and n are variables denoting natural numbers.

Would the rules not be ordered as they are actually laid out, the second rule would match any pair. Instead, it can be assumed that $n \neq 0$ when matching with it.

The last rule is to be tried last, as it matches all terms.

Transitive closures

We can easily see that all the compositions of rewrites starting with a natural number n end with the factorial of n :

$$n \xrightarrow{3} (n, 1) \xrightarrow{2} \dots \xrightarrow{2} (0, n!) \xrightarrow{1} n!, \quad \text{for } n \in \mathbb{N}.$$

Let us note $(\xrightarrow{n})$ the composition of $(\rightarrow)$ repeated $n - 1$ times:

$$\begin{aligned} (\xrightarrow{1}) &:= (\rightarrow) \\ (\xrightarrow{n+1}) &:= (\rightarrow) \circ (\xrightarrow{n}), \quad \text{with } n > 0. \end{aligned}$$

The **transitive closure** of $(\rightarrow)$ is defined as

$$(\twoheadrightarrow) := \bigcup_{i>0} (\xrightarrow{i}).$$

The factorial function coincides with the transitive closure of $(\rightarrow)$: $n \twoheadrightarrow n!$. Let $(\xrightarrow{*})$ be the reflexive-transitive closure of $(\rightarrow)$, that is,

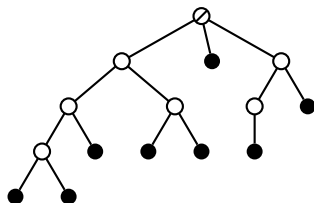
$$(\xrightarrow{*}) := (=) \cup (\twoheadrightarrow).$$

Functions

- A confluent system defines a **partial function**.
- If the system is also terminating, we get a **function**.
- We can name functions: for example, $c(1, d(n))$ is a term constructed with **function names** c and d , as well as variable n .
- A tuple tagged with a function name, like $f(x, y)$, is called a **function call**.
- The components of the tuples are then called **arguments**, for example $d(n)$ is the second argument of the call $c(1, d(n))$.
- It is possible for a function call to hold no arguments, like $d()$.
- We restrict the left-hand sides of rules to be function calls.

Trees

The topological understanding of a function call or a tuple is the finite **tree**. A tree is a hierarchical layout of information, for example:



The disks are called **nodes** and the segments which connect two nodes are called **edges**. The topmost node (with a diameter) is called the **root** and the bottommost ones (●) are called the **leaves**. All nodes except the leaves are seen to downwardly connect to some other nodes, called **children**. Upwardly, each node but the root is connected to another node, called its **parent**.

Trees

Trees are pervasive in informatics, especially in functional programming, but not only. Structured documents, like books, articles, and web pages, are composed of chapters, sections, paragraphs, figures, appendices, indices, etc.

The occurrences of these components are mutually constrained; for instance, it is understood that a section is part of a chapter and that appendices are located at the end of a document.

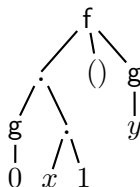
This hierarchical layout is meant to facilitate reading, and it supports the search for specific items of information.

When considering computer systems, these data must be uniformly encoded by means of a formal language.

Terms as trees

Trees can be used to depict terms as follows. A function call is a tree whose root is the function name and the children are the trees denoting the arguments.

A tuple can be considered as having an invisible function name represented by a node with a period (.) in the tree, in which case the components of the tuple are its children. For example,



is the tree corresponding to the term $f((g(0), (x, 1)), (), g(y))$.

Note that tuples, except the empty tuple, are not represented in the tree, since they encode the structure itself, which is already laid out. The number of arguments of a function is called **arity**.

Functional languages

- For programming, we only consider confluent systems because they define partial functions.
- To have only one reduction chain per term to the normal form, we may also enforce that rules are ordered.
- We also enforce that normal forms must be **values**, that is, terms without function calls or only with calls to functions not occurring on the left-hand sides.
- In other words, irreducible calls of defined functions are not allowed in values.
- Systems with these constraints are called **functional rewrite systems**.
- A program here is a term, and evaluating it means to rewrite the term
 1. to a value,
 2. or to end in error with a normal form that is not a value,
 3. or to never terminate.

Call-by-value

If a function call contains at least one other function call, there is a choice as to which one to reduce first.

One reduction strategy, named **call-by-value**, consists in always reducing the arguments before the call itself. Unfortunately, call-by-value enables otherwise terminating rewrites to not terminate. For instance, let us consider

$$f(x) \xrightarrow{\alpha} 0 \quad g() \xrightarrow{\beta} g()$$

We have $f(g()) \xrightarrow{\alpha} 0$ but $f(g()) \xrightarrow{\beta} f(g()) \xrightarrow{\beta} \dots$

Despite the loss of expressiveness, we shall retain in this lecture the call by value, as it is the strategy of OCaml and it is easy to follow.

In general, the order in which the arguments of a function call are evaluated is not specified.

Call-by-value

Another reason to choose call-by-value is that it allows us to restrict the shape of the left-hand sides, called **patterns**, to one, outermost function call.

For instance, we can then disallow, for being useless, a rule like

$$\text{plus}(x, \text{plus}(y, z)) \rightarrow \text{plus}(\text{plus}(x, y), z).$$

because, following call-by-value, the argument $\text{plus}(y, z)$ must be evaluated before the outermost call, therefore it cannot be part of any pattern.

Termination

We do not require by construction that all functional systems terminate, although that is a desirable property, but we nevertheless speak of ‘functions’ instead of ‘partial functions’ (which would be technically correct).

If we enforce termination, we loose **Turing-completeness**: we cannot program all computable functions.

The termination of rewrite systems and, in particular, functional programs, is an **undecidable problem** in general.

Nevertheless, there exists a set of well-known **decision criteria**, which we will use when programming recursive functions in OCaml.

Higher-order programs, recursion, λ -calculus

Most functional languages allow **higher-order function** definitions, whereas standard term-rewriting systems do not. For example:

$$f(g, 0) \rightarrow 1 \quad f(g, n) \rightarrow n \times g(g, n - 1) \quad \text{fact}(n) \rightarrow f(f, n)$$

where $n \in \mathbb{N}$. Note that these two definitions are **not** recursive, yet the function `fact` is the factorial function. A function definition is **recursive** if the name of the definition occurs on at least one right-hand side.

An adequate theoretical framework to understand higher-order functions is **λ -calculus**. In fact, λ -calculus features prominently in the semantics of programming languages, even not functional ones.

For didactic purposes, we began with rewrite systems because they offer implicit pattern matching (that is, rule selection).

Stacks in a functional setting

We show how to express linear data structures in a purely functional language and how to compute with them. The simplest of those structures is the **stack**.

A stack can be thought of as a finite series of items that can only be accessed sequentially from one end, called the **top**, following the analogy with a stack of material objects.

Stacks in a functional setting

To define a stack in a functional language, one uses calls to undefined functions to model all the possible shapes a stack can take.

Here, a stack can be either empty or not.

Let $\text{nil}()$ be the irreducible call that denotes an empty stack (that is, the function 'nil' is undefined).

If not empty, the stack then must contain a topmost (first) item, and a **substack**, that is, the stack made of all the items but the first.

Let $\text{cons}(x, s)$ denote the non-empty stack with the item x on top and the **immediate substack** s .

Inductive definition of stacks

- The previous definition is intuitive, but not formal yet.
- Data structures are usually defined **inductively**.
- Let $\mathcal{T}$ be the set of all possible terms and $\mathcal{S} \subset \mathcal{T}$ be the set of all stacks.
- Formally, $\mathcal{S}$ can be defined by **induction** as the smallest set such that
 1. $\text{nil}() \in \mathcal{S}$;
 2. if $x \in \mathcal{T}$ and $s \in \mathcal{S}$, then $\text{cons}(x, s) \in \mathcal{S}$.
- We take the smallest set because we do not want in $\mathcal{S}$ terms that were not constructed by the inductive rules (this is called the **smallest fixed point** of the inductive relation).

Appending a stack

- For all $s, t \in \mathcal{S}$, let us consider the functional program

$$\begin{aligned}\text{cat}(\text{nil}(), t) &\xrightarrow{\alpha} t \\ \text{cat}(\text{cons}(x, s), t) &\xrightarrow{\beta} \text{cons}(x, \text{cat}(s, t))\end{aligned}$$

- This is a functional program suitable for call-by-value reduction strategy, because the function calls in the patterns (that is, left-hand sides) are irreducible.
- The right-hand sides are stacks, so a call to 'cat' is a stack.
- Rule α says that, if the first stack is empty, the value is the second.
- In rule β , the first stack is not empty, and its top item is the top of the final stack (x), the immediate substack (s) being used in a call to 'cat' with the second stack (t) unchanged.

Appending a stack

This function appends the second stack at the bottom of a **copy** of the first. (The function name 'cat' stands for 'catenation', and is slightly better than 'app').

We could also say that this function appends a copy of the first stack on top of the other. But this is not the same as pushing the first stack on top of the other, like

$$\text{cat}(s, t) \rightarrow \text{cons}(s, t) \quad \text{Wrong!}$$

This is **not** appending: here, the stack s becomes an item of the resulting stack, but we want only the **contents** of s to be pushed on top of t , not their container.

Analogy: You may think of an empty stack as a empty cardboard box, open topside, and every item pushed in it is like a book that you put on the topmost book, or at the bottom of the box if this is the first book.

Appending a stack

Let us set the abbreviations

- $[] := \text{nil}()$,
- $x :: s := \text{cons}(x, s)$,

after the convention of the programming language OCaml.

We can further abbreviate

- $x_1 :: x_2 :: \dots :: x_n :: s := \text{cons}(x_1, \text{cons}(x_2, \dots \text{cons}(x_n, s)))$
- $[x] := x :: []$
- $[x_1; x_2, \dots; x_n] := x_1 :: x_2 :: x_3 :: \dots :: x_n :: []$.

Our system now becomes a bit more legible:

$$\begin{aligned} \text{cat}([], t) &\xrightarrow{\alpha} t \\ \text{cat}(x :: s, t) &\xrightarrow{\beta} x :: \text{cat}(s, t) \end{aligned}$$

Pattern matching

- Let us compute $\text{cat}([1; 2], [3])$.
- We need to find the first rule that is matched by this call.
- Remember that the arguments must be values before you do that.
- Intuitively, this means that we compare the call with the patterns, that is, the left-hand sides of the rules, in order.
- Here, rule α is not matched because $[1; 2] \neq []$.
- Rule β is matched because $[1; 2] = 1 :: [2]$ and equals the sub-pattern $x :: s$ if we assign $x := 1$ and $s := [2]$. Also, trivially, $t := [3]$ works.
- After the rule β is selected, the assignments are used to replace the variables in the right-hand sides.
- The whole evaluation is

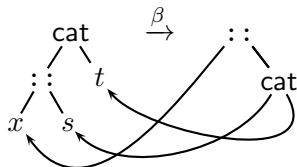
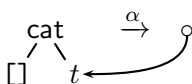
$$\text{cat}([1; 2], [3]) \xrightarrow{\beta} 1 :: \text{cat}([2], [3]) \xrightarrow{\beta} 1 :: 2 :: \text{cat}([], [3]) \xrightarrow{\alpha} [1; 2; 3].$$

Abstract Syntax Trees (ASTs)

- Depending on the context, we may use the arborescent depiction of terms to bring to the fore certain aspects of a computation.
- For example, it may be interesting to show how parts of the output (the right-hand side) are actually **shared** with the input (the left-hand side).
- In other words, how much (new) data is created by a given rule.
- This concept supposes that terms reside in some sort of space and that they can be referred to from different other terms.
- This abstract space serves as a model of an interpreter's memory, called the **heap**.

Data sharing

- Consider for instance



which is the same definition of 'cat' as given above:

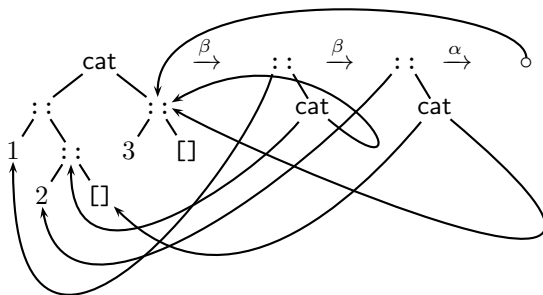
$$\text{cat}([], t) \xrightarrow{\alpha} t$$

$$\text{cat}(x::s, t) \xrightarrow{\beta} x::\text{cat}(s, t)$$

- The arrows on certain edges denote data sharing.
- When trees are used to visualise terms, they are called **abstract syntax trees** (AST).
- When some trees share subtrees, the whole **forest** (set of trees) is called a **directed acyclic graph** (DAG).

Data sharing and evaluation

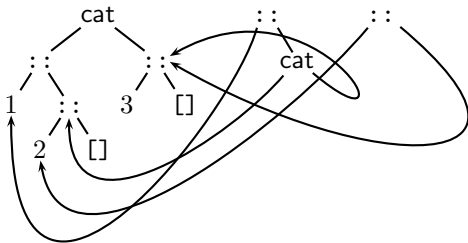
The evaluation of $\text{cat}([1; 2], [3])$ is:



Note that each right-hand side shows only the rewritten call, not the whole term.

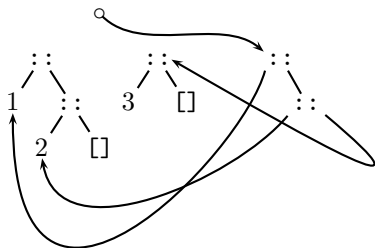
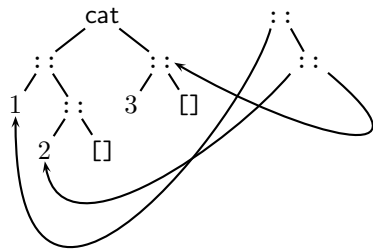
Data sharing and evaluation

Therefore, to reconstruct the value, we need to work leftwards and replace the rewritten call by the right-hand side:



We do not show the calls that have been replaced by their values.

Data sharing and evaluation



Call stack

We can see that the value of the second argument, [3], is shared with the value of the call, as well as the integers 1 and 2.

We saw how rewrites accumulated rightwards until the result had to be reconstructed by replacing leftwards the right-hand sides.

This dynamic is that of a stack, but a stack holding suspended function calls (that is, awaiting their values) and references to ASTs in the heap. This is the **call stack**.

The top of the stack (here, the rightmost abstract syntax tree) is the next term to be rewritten, unless the call on the left-hand side is the original call, in which case the top is its value.

Garbage collection

- In the abstract syntax trees (AST), the nodes with the calls that are replaced by their values have been immediately removed. In fact, they could have been removed later, in any order.
- The process which identifies and discards useless ASTs is called the **garbage collector** (GC).
- A term is useless if it cannot be accessed from anywhere in the call stack.
- Contrary to the heap, which can be thought of as an unordered set of trees (a forest of DAGs), the call stack is an ordered set that can be efficiently extended or reduced.
- The garbage collector can be conceived as a process interleaved with the process of evaluation.
- Like evaluators of functional programs, garbage collectors may also have different strategies (as to when dispose of useless data).

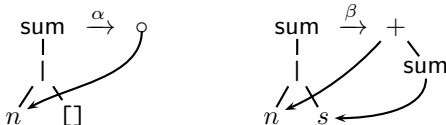
Tail call optimisation

Let us define a function that sums integers in a non-empty list:

$$\text{sum}[n] \xrightarrow{\alpha} n \quad \text{sum}(n::s) \xrightarrow{\beta} n + \text{sum}(s)$$

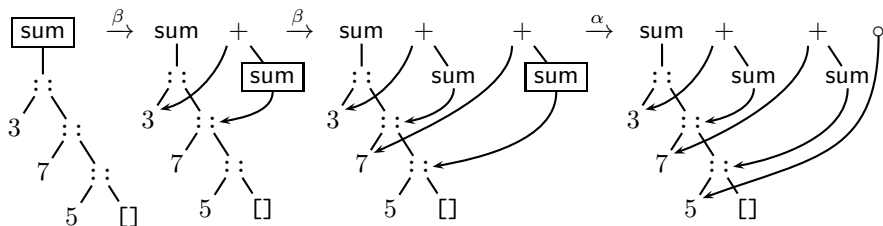
where we introduced a notation 'sum[n]' instead of 'sum([n])'. We will omit the parentheses when there is exactly one argument and it is clear when it starts and ends.

The view of the system with maximum sharing is:



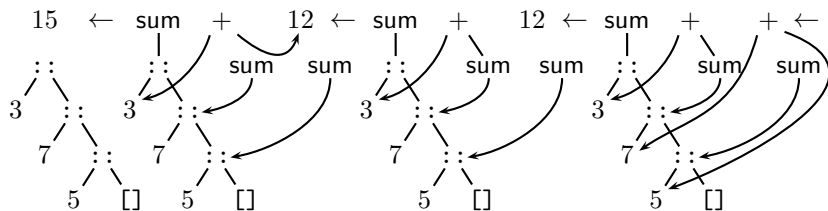
Tail call optimisation

The first phase of the evaluation of $\text{sum}([3; 7; 5])$ is:



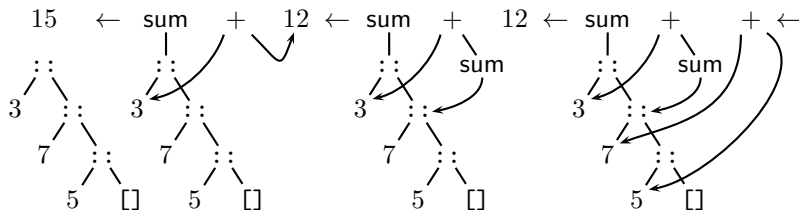
Tail call optimisation

The second phase (replacing the pending calls with their values) is



Tail call optimisation

The same, with a garbage collector that collects after each replacement:

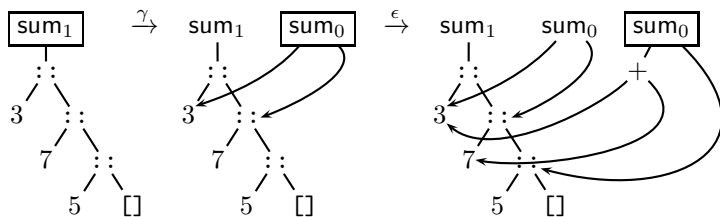


Tail call optimisation

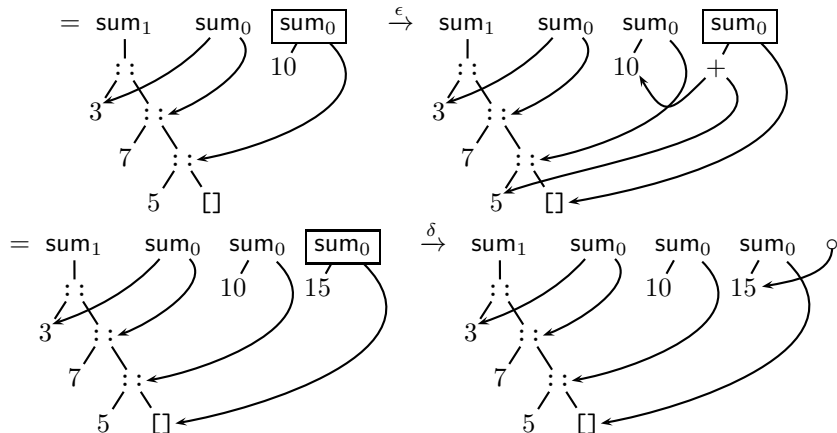
Let us consider an alternate version of 'sum':

$$\begin{aligned}\text{sum}_1(n :: t) &\xrightarrow{\gamma} \text{sum}_0(n, t). \\ \text{sum}_0(n, []) &\xrightarrow{\delta} n; \\ \text{sum}_0(n, m :: s) &\xrightarrow{\epsilon} \text{sum}_0(n + m, s).\end{aligned}$$

For $\text{sum}_1([3; 7; 5])$ we have the evaluation



Tail call optimisation

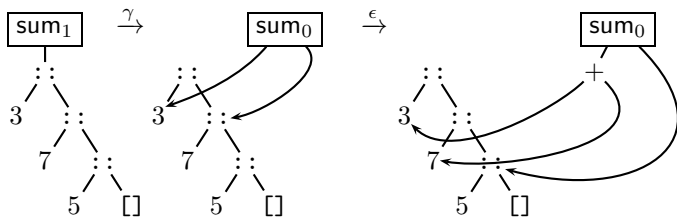


Tail call optimisation

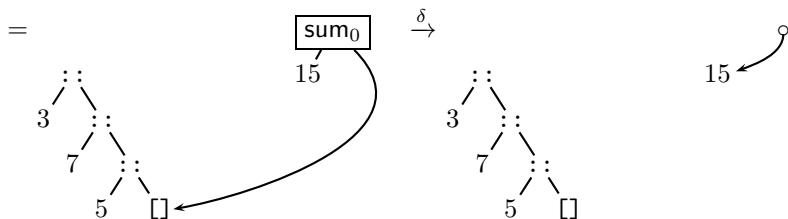
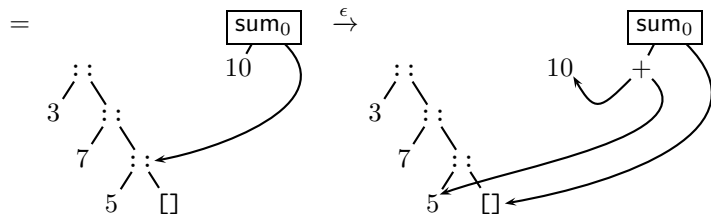
Notice that, in the case of sum_0 , we found the result (15) at the end of the first phase (of growing the call stack).

Therefore, there is no need for a second phase, which replaces the pending calls by their values: we can discard all those calls at once after the first phase.

Or, to see it even more clearly, let us rewrite the same first phase and assume that the collector reclaims immediately the useless pending calls:



Tail call optimisation



Tail call optimisation

As we see now, we can keep the size of the call stack constant, if we reclaim the top at each new rewrite.

This optimisation is called **tail call optimisation**.

Intuitively, a call in **tail position** is a call whose value is the value of the previously rewritten call.

For example, the call $f(x)$ in $1 + f(x)$ is not in tail position, because its value needs to be incremented.

Most compilers of functional languages detect and implement that optimisation that saves memory, including the C compiler of GCC.

Inductive proofs

Let us remark that

$$\text{cat}([1], [2; 3; 4]) \rightarrow [1; 2; 3; 4] \leftarrow \text{cat}([1; 2], [3; 4]).$$

It is enlightening to create **equivalence classes** of terms that are joinable. These classes then define an equivalence relationship ($\equiv$) as follows:

$a \equiv b$ if there exists a unique value v such that $a \xrightarrow{*} v$ and $b \xrightarrow{*} v$.

For instance, $\text{cat}([1; 2], [3; 4]) \equiv \text{cat}([1], [2; 3; 4])$.

The relation ($\equiv$) is indeed an equivalence because it is

- **reflexive:** $a \equiv a$;
- **symmetric:** if $a \equiv b$, then $b \equiv a$;
- **transitive:** if $a \equiv b$ and $b \equiv c$, then $a \equiv c$.

Inductive proofs

Of some interest are the following facts.

- If $x_1 \leftarrow x_2 \rightarrow x_3 \equiv x_4 \rightarrow x_5 \leftarrow x_6$, then $x_1 \equiv x_2 \equiv x_3 \equiv x_4 \equiv x_5 \equiv x_6$.
- If $f(x)$ and $f(y)$ have a value, then

$$x \equiv y \Rightarrow f(x) \equiv f(y),$$

that is, $(\equiv)$ is a **congruence relation**, as a consequence of confluence and termination.

If we want to prove equivalences with variables ranging over **infinite sets**, like $\text{cat}(s, \text{cat}(t, u)) \equiv \text{cat}(\text{cat}(s, t), u)$, we need the **induction principle**.

Well-founded induction

We define a **well-founded order** on a set A as being a binary relation ($\succ$) such that it is

- irreflexive: $\forall x \in A. \neg(x \succ x)$,
- transitive: $\forall x, y, z \in A. x \succ y \wedge y \succ z \Rightarrow x \succ z$,
- and does not have any **infinite descending chains**, to wit, there is no $a_0 \succ a_1 \succ a_2 \succ \dots$

The **well-founded induction principle** then states that, for any predicate $\mathbb{N}$ on a set A ,

$$\forall a \in A. \mathbb{N}(a) \text{ is implied by } \forall a. (\forall b. a \succ b \Rightarrow \mathbb{N}(b)) \Rightarrow \mathbb{N}(a).$$

When $A = \mathbb{N}$, this principle is called **mathematical (complete) induction**.

Well-founded induction

Structural induction is another particular case where $t \succ s$ holds if, and only if, s is a proper **subterm** of t , namely, the abstract syntax tree of s is included in the tree of t and $s \neq t$.

Sometimes, a restricted form of structural induction is enough. For instance, we can define $x :: s \succ s$, for any term x and any stack $s \in S$. Both x and s are **immediate subterms** of $x :: s$. (In other words, we dispensed with the transitivity axiom.) Because there are no infinite descending chains, any subset $B \subseteq A$ contains minimal elements $M \subseteq B$, that is, there is no $b \in B$ such that $a \succ b$, if $a \in M$. So

1. we have to prove $\mathbb{N}(a)$ for all $a \in M$, the so-called **basis**: here, only $\mathbb{N}([])$, as $[]$ is by construction the unique, minimal stacks, and,
2. assuming the so-called **induction hypothesis** $\mathbb{N}(s)$, we must prove $\mathbb{N}(x :: s)$ for all x .

Termination

When defining our functional rewrite systems, we allowed programs to not terminate (see pages 4 and 65).

We could actually have imposed some syntactic restrictions on recursive definitions in order to guarantee the termination of all functions. A well-known class of such terminating functions makes exclusively use of a bridled form of recursion called **primitive recursion**.

Unfortunately, many useful functions do not fit, easily or at all, in this framework and, as a consequence, most functional languages leave to the programmers the responsibility to check the termination of their programs. For theoretical reasons, it is not possible to provide a general criterion for termination, but some rules exist that cover many usages.

Termination is undecidable

Here is an informal argument that shows that there is no predicate on functions that decides where their subject terminates on all inputs. By contradiction, let us assume that there exists f such that

$$f(g) = \begin{cases} \text{true} & \text{if } g \text{ terminates on all inputs,} \\ \text{false} & \text{otherwise.} \end{cases}$$

Now let us define a function h as follows:

$$h(g) = \begin{cases} \text{loops forever} & \text{if } f(g), \\ \text{terminates on all inputs} & \text{otherwise} \end{cases}$$

In other words, h looks at $f(g)$ and does the opposite of what f predicts about g .

Termination is undecidable

Let us consider now $h(h)$:

- If $f(h)$, then, by definition of f , h terminates on all inputs, in particular $h(h)$ terminates. But, by definition of h , since $f(h)$ holds, $h(h)$ loops on all inputs, yielding a contradiction.
- If $f(h)$ is false, then, by definition of f , h does not terminate on all its inputs. But, by definition of h , since $f(h)$ does not hold, $h(h)$ terminates on all inputs, yielding another contradiction.

Termination / Factorial

Here is the definition of the factorial function as a functional rewrite system:

$$\begin{aligned}\text{fact } 0 &\rightarrow 1; \\ \text{fact } n &\rightarrow n \times \text{fact}(n - 1).\end{aligned}$$

Given a call $\text{fact}(n)$, with $n \geq 0$, the rewrites yield a series of calls $\text{fact}(n - 1)$, $\text{fact}(n - 2)$, etc. until $\text{fact}(0)$.

Termination here ensues from the **strictly decreasing argument** $n > n - 1 > n - 2 > \dots > 0$. (No infinite descent.)

Termination / Ackermann's function

Consider the following example where $m, n \in \mathbb{N}$:

$$\begin{aligned}\text{ack}(0, n) &\xrightarrow{\alpha} n + 1; \\ \text{ack}(m + 1, 0) &\xrightarrow{\beta} \text{ack}(m, 1); \\ \text{ack}(m + 1, n + 1) &\xrightarrow{\gamma} \text{ack}(m, \text{ack}(m + 1, n)).\end{aligned}$$

This is a simplified form of Ackermann's function, an early example of a total computable function which is not primitive recursive. It makes use of double recursion and two parameters to grow values as towers of exponents, for example,

$$\text{ack}(4, 3) \rightarrow 2^{2^{65536}} - 3.$$

It is not obviously terminating, because if the first argument does decrease, the second largely increases.

Termination / Ackermann's function

Let us define a well-founded ordering on pairs, called **lexicographic order**. Let $(\succ_A)$ and $(\succ_B)$ be well-founded orders on the sets A and B . Then $(\succ_{A \times B})$ defined as follows on $A \times B$ is well-founded:

$$(a_0, b_0) \succ_{A \times B} (a_1, b_1) :\Leftrightarrow a_0 \succ_A a_1 \text{ or } (a_0 = a_1 \text{ and } b_0 \succ_B b_1).$$

If $A = B = \mathbb{N}$ then $(\succ_A) = (\succ_B) = (>)$.

For example, $(\underline{1}, 7) \prec (\underline{2}, 3)$ and $(1, \underline{7}) \succ (1, \underline{3})$.

To prove that $\text{ack}(m, n)$ terminates for all $m, n \in \mathbb{N}$, first, we must find a well-founded order on the calls $\text{ack}(m, n)$, that is, the calls must be totally ordered without any infinite descending chain.

Here, a lexicographic order on $(m, n) \in \mathbb{N}^2$ extended to $\text{ack}(m, n)$ works:

$$\text{ack}(a_0, b_0) \succ \text{ack}(a_1, b_1) :\Leftrightarrow a_0 > a_1 \text{ or } (a_0 = a_1 \text{ and } b_0 > b_1).$$

Termination / Ackermann's function

Clearly, $\text{ack}(0, 0)$ is the minimum element.

Second, we must prove that 'ack' rewrites to smaller calls. We are only concerned with rules β and γ . With the former, we have the inequality

$$\text{ack}(m + 1, 0) \succ \text{ack}(m, 1).$$

With rule γ , we have the following inequalities:

$$\begin{aligned}\text{ack}(m + 1, n + 1) &\succ \text{ack}(m + 1, n), \\ \text{ack}(m + 1, n + 1) &\succ \text{ack}(m, p),\end{aligned}$$

for all values p , in particular when $\text{ack}(m + 1, n) \rightarrow p$. □

Implicitly, we used

$$x \succ y \Rightarrow f(x) \succ f(y),$$

that is, **evaluation is (strictly) order-preserving**.

Termination / Dependency pairs and subterm order

In general, the approach often taken to prove termination consists in finding a well-founded order ($\succ$) on the rewritten terms, which is entailed by the rewrite relation ($\rightarrow$), that is,

$$x \rightarrow y \Rightarrow x \succ y.$$

As we have seen, we do not always can order the whole terms x and y : only the calls in them to a given function are enough. These are called the **dependency pairs**. Calls to other functions can be assumed to be terminating.

With the factorial and Ackermann's function, we have seen how to deal with one or more integer arguments to these calls.

What about other data types, like lists and trees?

Termination / Dependency pairs and subterm order

One well-founded order for lists (stacks) is the **immediate subterm order**, satisfying

$$x :: s \succ s \quad \text{and} \quad x :: s \succ x.$$

The immediate subterm order simply means that a list is 'greater' than its first element and the rest of the list (called the **tail**). It is a special case of the **proper subterm order**, in which a structured value is 'greater' than any of its components, wherever they are located.

These orders are convenient to prove the termination of calls to functions whose definitions are recursive in terms of substructures of their arguments: this is quite a common case.

For example, let us recall the function concatenating a list to another:

$$\begin{aligned} \text{cat}([], t) &\xrightarrow{\alpha} t \\ \text{cat}(x :: s, t) &\xrightarrow{\beta} x :: \text{cat}(s, t) \end{aligned}$$

We simply have: $x :: s \succ s \Rightarrow \text{cat}(x :: s, t) \succ \text{cat}(s, t)$.

Termination / Insertion sort

Let us consider **insertion sort**:

$$\begin{array}{ll} \text{isort } [] \xrightarrow{\alpha} []; & \text{ins}(x, y :: s) \xrightarrow{\gamma} y :: \text{ins}(x, s), \text{ if } x \succ y; \\ \text{isort}(x :: s) \xrightarrow{\beta} \text{ins}(x, \text{isort } s). & \text{ins}(x, s) \xrightarrow{\delta} x :: s. \end{array}$$

Let us consider a short example:

$$\begin{aligned} \text{isort } [3, 1, 2] &\xrightarrow{\beta} \text{ins}(3, \text{isort } [1; 2]) \\ &\xrightarrow{\beta} \text{ins}(3, \text{ins}(1, \text{isort } [2])) \\ &\xrightarrow{\beta} \text{ins}(3, \text{ins}(1, \text{ins}(2, \text{isort } []))) \\ &\xrightarrow{\alpha} \text{ins}(3, \text{ins}(1, \text{ins}(2, []))) \\ &\xrightarrow{\delta} \text{ins}(3, \text{ins}(1, [2])) \\ &\xrightarrow{\delta} \text{ins}(3, [1; 2]) \\ &\xrightarrow{\gamma} 1 :: \text{ins}(3, [2]) \\ &\xrightarrow{\gamma} 1 :: 2 :: \text{ins}(3, []) \\ &\xrightarrow{\delta} 1 :: 2 :: 3 :: [] \\ &= [1; 2; 3]. \end{aligned}$$

Termination / Insertion sort

Informally, what soundness means is that, if some program terminates, then the result is what was expected. This property is called **partial correctness** when it is relevant to distinguish it from **total correctness**, which is partial correctness and termination.

Let us prove the termination of 'isort' by the method of dependency pairs. The pairs to order are the following:

- $(\text{ins}(x, y :: s), \text{ins}(x, s))_\gamma$,
- $(\text{isort}(x :: s), \text{isort } s)_\beta$ and
- $(\text{isort}(x :: s), \text{ins}(x, \text{isort } s))_\beta$.

Termination / Insertion sort

By using the proper subterm relation on the second parameter of 'ins', we order the first pair:

$$y :: s \succ s \Rightarrow \text{ins}(x, y :: s) \succ \text{ins}(x, s).$$

This is enough to prove that 'ins' terminates. The second pair is similarly oriented:

$$x :: s \succ s \Rightarrow \text{isort}(x :: s) \succ \text{isort } s.$$

Termination / Insertion sort

The third pair is not worth considering after all, because we already know that 'ins' terminates, so the second pair is enough to entail the termination of 'isort'.

In other words, since 'ins' terminates, it can be considered, as far as termination analysis is concerned, as a data constructor, so the third pair becomes:

$$x :: s \succ s \Rightarrow \text{isort}(x :: s) \succ \text{isort } s \Rightarrow \text{isort}(x :: s) \succ \underline{\text{ins}}(x, \text{isort } s).$$

where ins stands for 'ins' considered as a **constructor**, that is, an undefined function. □

Termination / List flattening

Let us try a more complicated example:

$$\begin{aligned}\text{flat } [] &\xrightarrow{\alpha} [] \\ \text{flat}([] :: t) &\xrightarrow{\beta} \text{flat } t \\ \text{flat}((x :: s) :: t) &\xrightarrow{\gamma} \text{cat}(\text{flat}(x :: s), \text{flat } t) \\ \text{flat}(y :: t) &\xrightarrow{\delta} y :: \text{flat } t\end{aligned}$$

Some evaluations:

$$\begin{aligned}\text{flat } [] &\twoheadrightarrow [] \\ \text{flat } [[]]; [[]] &\twoheadrightarrow [] \\ \text{flat } [[]]; [1; [2; []]]; 3; [] &\twoheadrightarrow [1; 2; 3]\end{aligned}$$

What 'flat' does is flatten a list of lists arbitrarily deep.

Termination / List flattening

A detailed evaluation:

$$\begin{aligned} \text{flat } [[]; [[1]; 2]; 3] &\xrightarrow{\beta} \text{flat } [[[1]; 2]; 3] \\ &\xrightarrow{\gamma} \text{cat}(\text{flat } [[1]; 2], \text{flat } [3]) \\ &\xrightarrow{\gamma} \text{cat}(\text{cat}(\text{flat } [1], \text{flat } [2]), \text{flat } [3]) \\ &\xrightarrow{\delta} \text{cat}(\text{cat}(1 :: \text{flat } [], \text{flat } [2]), \text{flat } [3]) \\ &\xrightarrow{\alpha} \text{cat}(\text{cat}([1], \text{flat } [2]), \text{flat } [3]) \\ &\xrightarrow{\delta} \text{cat}(\text{cat}([1], 2 :: \text{flat } []), \text{flat } [3]) \\ &\xrightarrow{\alpha} \text{cat}(\text{cat}([1], [2]), \text{flat } [3]) \\ &\xrightarrow{\delta} \text{cat}(\text{cat}([1], [2]), 3 :: \text{flat } []) \\ &\xrightarrow{\alpha} \text{cat}(\text{cat}([1], [2]), [3]) \\ &\Rightarrow [1; 2; 3]. \end{aligned}$$

Termination / List flattening

The function 'cat' is independent of 'flat' and we proved its termination in isolation by using the proper subterm order on its first argument.

Assuming now that 'cat' terminates, let's prove the termination of 'flat'.

Because the recursive calls to 'flat' contain only (list) constructors, we can try to order their arguments.

The proper subterm order works:

$$\begin{array}{ll} y :: t \succ t & \text{(rule } \delta \text{ and rule } \beta \text{ when } y = []) \\ (x :: s) :: t \succ t & \text{(rule } \gamma) \\ (x :: s) :: t \succ x :: s & \text{(rule } \gamma) \end{array}$$

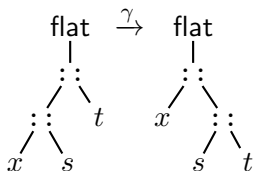
Termination ensues. □

Termination / List flattening

Let us consider now an alternative design for rule γ :

$$\begin{array}{lcl} \text{flat} [] & \xrightarrow{\alpha} & [] \\ \text{flat}([] :: t) & \xrightarrow{\beta} & \text{flat } t \\ \text{flat}((x :: s) :: t) & \xrightarrow{\gamma} & \boxed{\text{flat}(x :: s :: t)} \\ \text{flat}(y :: t) & \xrightarrow{\delta} & y :: \text{flat } t \end{array}$$

This amounts to perform a **right rotation** of the abstract syntax tree of the term:



Termination / Measures

Here, the proper subterm order fails:

$$(x :: s) :: t \not\prec x :: (s :: t) = x :: s :: t$$

because $t \not\prec s :: t$.

We need another method to prove termination: **measures**.

A measure $\mathcal{M}[\![\cdot]\!]$ is a map from terms to a well-ordered set $(A, \succ)$, which is **monotonically decreasing** with respect to the rewrite relation:

$$x \rightarrow y \Rightarrow \mathcal{M}[\![x]\!] \succ_A \mathcal{M}[\![y]\!].$$

Actually, as earlier, we will only consider **dependency pairs**, that is, pairs of calls whose first component is the left-hand side of a rule and the second components are the calls in the right-hand side of the same rule.

Termination / Polynomial measures

The pairs are

1. $(\text{flat}([] :: t), \text{flat } t)_{\beta}$
2. $(\text{flat}((x :: s) :: t), \text{flat}(x :: s :: t))_{\gamma}$
3. $(\text{flat}(y :: t), \text{flat } t)_{\delta}$

with $y \notin S$, that is, y is not a list.

We can actually drop the function names, as all the pairs involve ‘flat’.

A common class of measures are monotonically increasing embeddings into $(\mathbb{N}, >)$, so let us seek a measure satisfying:

$$\begin{aligned} \mathcal{M}[(x :: s) :: t] &> \mathcal{M}[x :: s :: t] \\ \mathcal{M}[y :: t] &> \mathcal{M}[t], \quad \text{if } y \notin S \text{ or } y = []. \end{aligned}$$

Termination / Polynomial measures

For instance, let us set the following **polynomial measure**:

$$\begin{aligned}\mathcal{M}[x::s] &:= 1 + 2 \cdot \mathcal{M}[x] + \mathcal{M}[s] \\ \mathcal{M}[y] &:= 0, && \text{if } y \notin S \text{ or } y = [].\end{aligned}$$

We have

$$\begin{aligned}\mathcal{M}[(x::s)::t] &= 3 + 4 \cdot \mathcal{M}[x] + 2 \cdot \mathcal{M}[s] + \mathcal{M}[t] \\ \mathcal{M}[x::s::t] &= 2 + 2 \cdot \mathcal{M}[x] + 2 \cdot \mathcal{M}[s] + \mathcal{M}[t]\end{aligned}$$

Then

$$\begin{aligned}\mathcal{M}[(x::s)::t] &> \mathcal{M}[x::s::t] \\ \mathcal{M}[y::t] = 1 + \mathcal{M}[t] &> \mathcal{M}[t]\end{aligned}$$

Since $\mathcal{M}[x] \in \mathbb{N}$, for all x , these inequalities entail the termination of ‘flat’ (no infinite descent). □

Termination / Dependency pairs in more details

Let's try to be a little bit more rigorous with dependency pair analysis.

Perhaps the first thing we have noticed is the use of conditions (guards) on rewrite rules. They are actually simple ways to make the systems more readable and more efficiently implemented in functional languages. In a way, they are akin to macros.

For example, consider the definitions of functions calculating the maximum and the minimum of two numbers:

$\max(0, y) \rightarrow y;$	$\min(0, y) \rightarrow 0;$
$\max(x, 0) \rightarrow x;$	$\min(x, 0) \rightarrow 0;$
$\max(x, y) \rightarrow 1 + \max(x-1, y-1).$	$\min(x, y) \rightarrow 1 + \min(x-1, y-1).$

Then insertion sort can be written (very inefficiently) as:

$$\begin{aligned} \text{ins}(x, []) &\rightarrow [x]; \\ \text{ins}(x, y :: s) &\rightarrow \min(x, y) :: \text{ins}(\max(x, y), s). \end{aligned}$$

Termination / Dependency pairs in more details

An equivalent but more compact definition would determine maximum and the minimum at the same time, like so:

$$\begin{aligned} \text{isort } [] &\xrightarrow{\alpha} []; \\ \text{isort}(x :: s) &\xrightarrow{\beta} \text{ins}(x, \text{isort } s). \\ \text{ins}(x, []) &\xrightarrow{\gamma} [x]; \\ \text{ins}(x, y :: s) &\xrightarrow{\delta} \text{choose}(x, y :: s, x, y). \\ \text{choose}(x, y :: s, \text{succ } a, \text{succ } b) &\xrightarrow{\epsilon} \text{choose}(x, y :: s, a, b); \\ \text{choose}(x, y :: s, a, 0) &\xrightarrow{\phi} y :: \text{ins}(x, s); \\ \text{choose}(x, y :: s, 0, \text{succ } b) &\xrightarrow{\psi} x :: y :: s. \end{aligned}$$

Here, we use Peano's unary encoding of integers:

- the successor function 'succ', so that $\text{succ}(x)$ denotes canonically the value of $x + 1$; (We do not have to assume '-'.)
- Also, '0' here actually stands for $\text{zero}()$.

Termination / Dependency pairs in more details

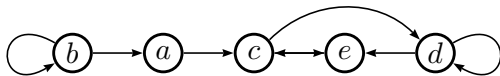
Instead of denoting the function symbols that are considered undefined in a dependency pair, as we did before, we are going to mark the defined ones (that is equivalent), with a sharp symbol ($\#$). This enables to write **the pairs as rewrite rules**, which is more readable.

We have five of them:

$$\begin{array}{lll} \text{isort}^\#(x :: s) \xrightarrow{a} \text{ins}^\#(x, \text{isort } s), & \text{from } \beta; \\ \text{isort}^\#(x :: s) \xrightarrow{b} \text{isort}^\# s, & \text{from } \beta; \\ \text{ins}^\#(x, y :: s) \xrightarrow{c} \text{choose}^\#(x, y :: s, x, y), & \text{from } \delta; \\ \text{choose}^\#(x, y :: s, \text{succ } a, \text{succ } b) \xrightarrow{d} \text{choose}^\#(x, y :: s, a, b), & \text{from } \epsilon; \\ \text{choose}^\#(x, y :: s, a, 0) \xrightarrow{e} \text{ins}^\#(x, s), & \text{from } \phi. \end{array}$$

Termination / Dependency pairs in more details

We can visualise better the pairs by drawing the dependency graph: each vertex is labelled by a pair $(a, b, c \text{ etc.})$, and there is an arc $i \rightarrow j$ if the right-hand side of pair i matches the left-hand side of pair j :



That graph contains two **strongly-connected components (SCCs)**: $\{b\}$ and $\{c, d, e\}$. We must show that **all cycles in each SCC map to a strictly decreasing measure**.

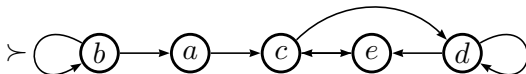
The first one is easy (it is a loop):

$$x :: s \succ s \Rightarrow \text{isort}^\#(x :: s) \xrightarrow{b} \text{isort}^\# s$$

Another way to put it is that the projection of the pair b on the parameter of $\text{isort}^\#$, that is $x :: s \rightarrow s$, if ordered, implies that the original pair is ordered too.

Termination / Dependency pairs in more details

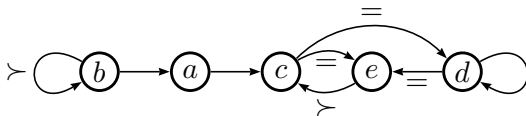
We can annotate the graph with the ordering relation we found:



For the second and last SCC $\{c, d, e\}$, if we project $\text{ins}^\#$ and choose $^\#$ on their second parameter, we get:

$$\begin{aligned} y :: s &\xrightarrow{c} y :: s, \\ y :: s &\xrightarrow{d} y :: s, \\ y :: s &\xrightarrow{e} s. \end{aligned}$$

So all paths through e strictly decreases the second parameter:



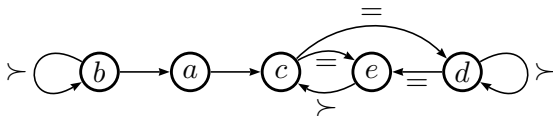
Termination / Dependency pairs in more details

Now the SCC $\{c, d, e\}$ reduces to the loop SCC $\{d\}$.

Pair d is easy to order: for example, we can project the components of the pair along the third parameter of $\text{choose}^\#$:

$$\text{succ } a \xrightarrow{d} a.$$

So



(Remember that the immediate subterm ordering ($\succ$) applies to the result of different projections.)

This entails that the system is terminating.



Proving associativity of stack concatenation

The definition of the concatenation of two stacks was:

$$\text{cat}([], t) \xrightarrow{\alpha} t; \quad \text{cat}(x :: s, t) \xrightarrow{\beta} x :: \text{cat}(s, t).$$

Let us prove the associativity of 'cat', which we express formally as

$$\text{CatAssoc}(s, t, u): \text{cat}(s, \text{cat}(t, u)) \equiv \text{cat}(\text{cat}(s, t), u)$$

for all stack values s , t and u .

Proving associativity of stack concatenation

We apply the well-founded induction principle to the structure of s , so we must establish

- the basis $\forall t, u \in S. \text{CatAssoc}([], t, u)$, and the
- step $\forall s, t, u \in S. \text{CatAssoc}(s, t, u) \Rightarrow \forall x \in T. \text{CatAssoc}(x :: s, t, u)$.

Underlining the call being rewritten, the base case is:

$$\underline{\text{cat}}([], \text{cat}(t, u)) \xrightarrow{\alpha} \text{cat}(t, u) \xleftarrow{\alpha} \text{cat}(\underline{\text{cat}}([], t), u).$$

Let us assume now $\text{CatAssoc}(s, t, u)$, called the **induction hypothesis**, and let us prove $\text{CatAssoc}(x :: s, t, u)$, for any term x .

Proving associativity of stack concatenation

The goal here is to use the rewrite system as an abstract machine to prove a property, with the timely help of the induction principle.

Precisely, we want to rewrite each side of the equivalence we wish to prove until we either find the same term (equality), or we use the induction hypothesis (equivalence).

Since we are going to work with equivalences, we should lift the call-by-value reduction strategy here, as it does not matter when arguments are evaluated, as long as their evaluation terminates and is confluent: we always assume these properties on the system hold when proving properties of the functions it defines.

Proving associativity of stack concatenation

We may start with the left-hand side of $\text{CatAssoc}(x :: s, t, u)$, that is $\text{cat}(x :: s, \text{cat}(t, u))$.

We notice that this is **not** a value, as the call contains a reducible call, $\text{cat}(t, u)$.

When using functions to compute, we match a call containing only values against the patterns of the rules.

Here, we would like to match a pattern against a pattern. This is more general than matching, because values are patterns, and is called **unification**. Given two patterns, we want to find whether we can find assignments to the variables of these so that, when they are substituted by their denotation, the two patterns become equal.

Proving associativity of stack concatenation

Here, we would like to unify $\text{cat}(x :: s, \text{cat}(t, u))$ (called the **goal**) and $\text{cat}(x :: s, t)$ (the pattern).

The problem is that we have variables that are the same in the goal and the pattern (x , s and t).

The case of $x :: s$ and $x :: s$ is harmless, but the unification of $\text{cat}(t, u)$ and t is not.

That is why it is best to rename the variables in the pattern so they are not found in the goal (this is the α -**renaming** of λ -calculus), and then try to unify them.

To simplify things even more, we are going to rename *all* the variables in the definition of 'cat'.

Proving associativity of stack concatenation

Let us try for example the following renaming:

$$\begin{aligned}\text{cat}([], b) &\xrightarrow{\alpha} b \\ \text{cat}(i :: a, b) &\xrightarrow{\beta} i :: \text{cat}(a, b)\end{aligned}$$

We want to unify the goal $\text{cat}(x :: s, \text{cat}(t, u))$ with the pattern $\text{cat}(i :: a, b)$.

The unification yields the equations $x = i$, $s = a$ and $\text{cat}(t, u) = b$.

Then we apply the set of equations from the unification, so the right-hand side of β , $i :: \text{cat}(a, b)$, becomes $x :: \text{cat}(s, \text{cat}(t, u))$.

Proving associativity of stack concatenation

We have:

$$\begin{aligned} \underline{\text{cat}}(x :: s, \text{cat}(t, u)) &\xrightarrow{\beta} x :: \text{cat}(s, \text{cat}(t, u)) \\ &\equiv x :: \text{cat}(\text{cat}(s, t), u) && \text{CatAssoc}(s, t, u) \\ &\xleftarrow{\beta} \underline{\text{cat}}(x :: \text{cat}(s, t), u) \\ &\xleftarrow{\beta} \text{cat}(\underline{\text{cat}}(x :: s, t), u). \end{aligned}$$

Note the use of the inductive hypothesis $\text{CatAssoc}(s, t, u)$.

This derivation above establishes that $\text{CatAssoc}(x :: s, t, u)$ holds.

The induction principle entails that $\forall s, t, u \in S. \text{CatAssoc}(s, t, u)$. □

Unification

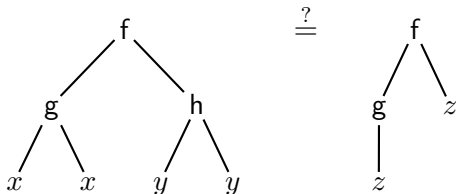
The best way to think about unification is to think of terms as trees, as we did page 61. A function is a subtree and a constant is a leaf.

The unification algorithm is then very simple:

- if the two terms are constants, they match if they are verbatim the same;
- if one of the terms is a variable, then they match;
- if the two terms are functions, they match if and only if
 - they are the same (name),
 - they have the same **arity** (number of arguments, that is, number of subterms, or number of subtrees),
 - if they have arguments, the n -th argument of one term matches the n -th argument of the other.

Unification / Example #1

For example, let's unify the terms $f(g(x, x), h(y, y))$ and $f(g(z), z)$.

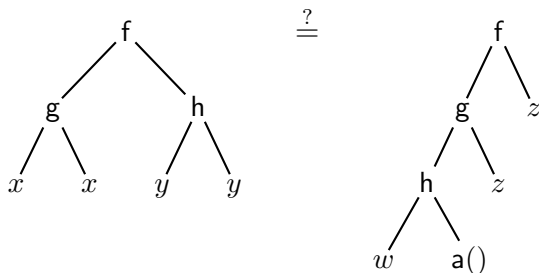


They do not unify because the subterm $g(x, x)$ does not match $g(z)$: the arity of g is different.

Note how the terms are non-linear, even though we do not allow them in our systems. Indeed, unification is all about equalities, which non-linearity is about. So let's allow it for exercises.

Unification / Example #2

Consider now the terms $f(g(x, x), h(y, y))$ and $f(g(h(w, a()), z), z)$:



Unification / Example #2

The steps are

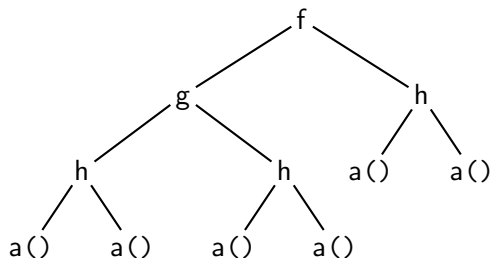
$x = h(w, a())$	$x = h(w, a())$	$h(w, a()) = h(y, y)$	$w = y$
$x = z$	$x = h(y, y)$	$x = h(y, y)$	$x = h(y, y)$
$z = h(y, y)$	$z = x$	$z = x$	$y = a()$
			$z = x$

Finally we obtain the **substitution**

$$\begin{aligned}w &= a() \\x &= h(a(), a()) \\y &= a() \\z &= h(a(), a())\end{aligned}$$

Unification / Example #2

This answer means that if we take the two trees of the query and replace the variables according to the substitution, we get the exact same tree. Indeed, we get



More proofs by induction

Here is again the concatenation of two stacks:

$$\begin{aligned}\text{cat}([], t) &\xrightarrow{\alpha} t \\ \text{cat}(x :: s, t) &\xrightarrow{\beta} x :: \text{cat}(s, t)\end{aligned}$$

Here is a functional rewrite system that defines a function 'len' that computes the length of a stack.

$$\begin{aligned}\text{len} [] &\xrightarrow{\gamma} 0; \\ \text{len}(x :: s) &\xrightarrow{\delta} 1 + \text{len } s.\end{aligned}$$

So, for example: $\text{len} [1; 2; 3] \rightarrow 1 + (1 + (1 + (0))) = 3$.

Let us prove the property

$$\text{LenCat}(s, t): \text{len}(\text{cat}(s, t)) \equiv \text{len } s + \text{len } t.$$

That is: concatenation preserves the length of the stacks.

More proofs by induction

Note how there are **two parameters**, s and t , instead of one.

In this case, we can try to prove the property by induction on one parameter, as we did before, and hope that the induction hypothesis with the other parameter considered constant suffice. Let us chose the first one.

The base case is $\forall t \in S. \text{LenCat}([], t)$, that is

$$\text{len}(\text{cat}([], t)) \equiv \text{len} [] + \text{len } t.$$

It is immediate (the call being rewritten is underlined):

$$\text{len}(\underline{\text{cat}}([], t)) \xrightarrow{\alpha} \text{len } t = 0 + \text{len } t \xleftarrow{\gamma} \underline{\text{len}} [] + \text{len } t.$$

More proofs by induction

The general case $\forall s, t \in S. \text{LenCat}(s, t) \Rightarrow \forall x \in T. \text{LenCat}(x :: s, t)$, where S is the set of all stacks, and T is the set of all terms ($S \subset T$).

Let us assume now $\forall s, t \in S. \text{LenCat}(s, t)$ (the induction hypothesis), and prove $\forall x \in T. \text{LenCat}(x :: s, t)$.

We have:

$$\begin{aligned} \text{len}(\underline{\text{cat}}(x :: s, t)) &\xrightarrow{\beta} \underline{\text{len}}(x :: \text{cat}(s, t)) \\ &\xrightarrow{\delta} 1 + \text{len}(\text{cat}(s, t)) \\ &\equiv 1 + (\text{len } s + \text{len } t) \\ &= (1 + \text{len } s) + \text{len } t \\ &\xleftarrow{\delta} \underline{\text{len}}(x :: s) + \text{len } t. \end{aligned} \quad \begin{array}{l} \text{LenCat}(s, t) \\ \text{associativity of } (+) \end{array}$$

We proved $\forall x \in T. \text{LenCat}(x :: s, t)$. □

Higher-order rewrite systems

We already extended term-rewriting systems (TRS) with conditions.

We can also extend them in another direction: variables could denote functions. These are called **higher-order systems**.

They enable defining functions that can be parameterised by other functions.

For example, let us consider **iterators**. Typically, those are functions that traverse a data structure and apply a given function at each step.

Maps on stacks

In the case of stacks, a simple iterator is a function 'map' that takes a stack and returns a stack made of the images of the elements, in the original order. So $[e_1; e_2; \dots; e_n]$ becomes $[f(e_1); f(e_2); \dots; f(e_n)]$.

$$\begin{aligned}\text{map}(f, []) &\xrightarrow{\alpha} []; \\ \text{map}(f, x :: s) &\xrightarrow{\beta} f(x) :: \text{map}(f, s).\end{aligned}$$

For example, if we set

$$\begin{aligned}\text{incr}(n) &\rightarrow n + 1. \\ \text{square}(n) &\rightarrow n * n.\end{aligned}$$

then we have

$$\begin{aligned}\text{map}(\text{incr}, [4; 4; 5; 0]) &\rightarrow [5; 5; 6; 1]. \\ \text{map}(\text{square}, [4; 4; 5; 0]) &\rightarrow [16; 16; 25; 0].\end{aligned}$$

Folds on stacks

Folds take an additional parameter which is supposed to be the initial value of an **accumulator**. In this framework, the given function is applied to an element of the stack *and to the current value of the accumulator*.

For example, for summing all integers in a stack, we would have

$$\text{fold}(\text{add}, [4; 4; 5; 0], 0) \rightarrow 13,$$

where $\text{add}(x, y) \rightarrow x + y$.

Note how the direction of iteration does not matter, because

$$4 + (4 + (5 + (0 + \underline{0}))) = (((\underline{0} + 4) + 4) + 5) + 0,$$

where the initial value of the accumulator is 0.

Lambdas

Note how cumbersome it is sometimes to have to define the function applied by the iterator (here a fold), when it is probably used only once.

To remedy this, we can further extend our systems with **anonymous functions**, also known as **lambdas** (a name coming from the λ -calculus, which is a different framework in which to model computations.)

For example, we will write **fun** $x \rightarrow x$ to denote the identity function, without naming it. Of course, we have to forbid calling any function 'fun' now (in other words, '**fun**' is a **keyword**).

In our case above:

$$\text{fold}((\text{fun } (x, y) \rightarrow x + y), [4; 4; 5; 0], 0) \rightarrow 13,$$

A map is a fold

A map is then a special case of a fold, where the accumulator is the new stack:

$$\text{map}(f, s) \rightarrow \text{fold}((\text{fun } (e, a) \rightarrow f(e) :: a), s, []),$$

where a is the accumulator and e a stack element.

Contrary to $(+)$, pushing on a stack is not a symmetric relation, so the order of traversal matters.

Here, since we want to conserve the order of the original elements of the stack, the fold must apply the given function first to the bottommost element, then the penultimate, then the antepenultimate etc. until the top. If we consider a stack a being layout horizontally, this means applying the function from right to left.

This is called a **right fold**.

Right fold

Let's rename 'fold' as 'foldr' ('fold right'). Then:

$$\begin{aligned}\text{foldr}(f, [], a) &\rightarrow a; \\ \text{foldr}(f, x :: s, a) &\rightarrow f(x, \text{foldr}(f, s, a)).\end{aligned}$$

A way to understand this definition is by extension:

$$\text{foldr}(f, [e_1; e_2; \dots; e_n], a) \equiv f(e_1, f(e_2, \dots f(e_n, a))).$$

Note that the recursive call in the definition of 'foldr' is not in tail position. (See page 81 and onwards.)

In this case, this means that all the calls to the iterated function f will be waiting for all the calls to 'foldr' to be finished (as well as all the calls to f on the previous elements of the stack).

Generally speaking, the call stack will grow at least proportionally to the length of the stack.

Left fold

The dual iteration of a right fold is a **left fold**:

$$\text{foldr}(f, [e_1; e_2; \dots; e_n], a) \equiv f(e_n, \dots f(e_2, f(e_1, a))).$$

It is a simple matter of rearranging the calls in the definition of 'foldr':

$$\begin{aligned}\text{foldl}(f, [], a) &\rightarrow a; \\ \text{foldl}(f, x :: s, a) &\rightarrow \text{foldl}(f, s, f(x, a)).\end{aligned}$$

Note how the recursive call is in tail position, so, if tail call optimisation is performed, the call stack will grow a constant a constant amount if the calls to the given function also grow the stack a constant amount (in total).

These considerations on memory usage are only meaningful when expressing our systems in programming languages and running the programs, but not when designing a system.

Inference rules

Let's say we want to define a function 'cut' that takes a stack s and an index k in that stack (the top has index 0) and returns the prefix of the stack s of length k (if any) and the corresponding suffix.

For instance,

$$\begin{aligned}\text{cut}([4; 2], 0) &\rightarrow ([], [4; 2]). \\ \text{cut}([5; 3; 6; 0; 2], 3) &\rightarrow ([5; 3; 6], [0; 2]).\end{aligned}$$

but, for the sake simplicity, nothing is said about $\text{cut}([0], 7)$ and $\text{cut}([0], -1)$, which we write as stuck terms:

$$\begin{aligned}\text{cut}([0], 7) &\nrightarrow \\ \text{cut}([0], -1) &\nrightarrow\end{aligned}$$

We have:

$$\begin{aligned}\text{cut}(s, 0) &\rightarrow ([], s); & \text{push}(x, (t, u)) &\rightarrow (x :: t, u). \\ \text{cut}(x :: s, k + 1) &\rightarrow \text{push}(x, \text{cut}(s, k)).\end{aligned}$$

Inference rules

When the value of a recursive call needs to be deconstructed, it is convenient to use an extension of our language to avoid creating auxiliary functions like ‘push’:

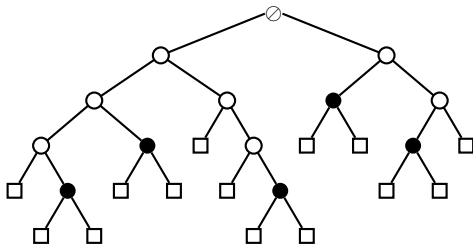
$$\text{cut}(s, 0) \rightarrow ([], s) \quad \text{NIL} \qquad \frac{\text{cut}(s, k) \rightarrow (t, u)}{\text{cut}(x :: s, k + 1) \rightarrow (x :: t, u)} \text{P}_{\text{REF}}$$

The new construct is called an *inference rule* because it means: ‘For the value of $\text{cut}(x :: s, k + 1)$ to be $(x :: t, u)$, we infer that the value of $\text{cut}(s, k)$ must be (t, u) ’. This interpretation corresponds to an upwards reading of the rule P_{REF} (*prefix*). Just as we compose horizontally rewrite rules, we compose inference rules vertically, stacking them, as in

$$\frac{\frac{\frac{\text{cut}([0; 2], 0) \rightarrow ([], [0; 2])}{\text{cut}([6; 0; 2], 1) \rightarrow ([6], [0; 2])}}{\text{cut}([3; 6; 0; 2], 2) \rightarrow ([3; 6], [0; 2])}}{\text{cut}([5; 3; 6; 0; 2], 3) \rightarrow ([5; 3; 6], [0; 2])}$$

Binary trees

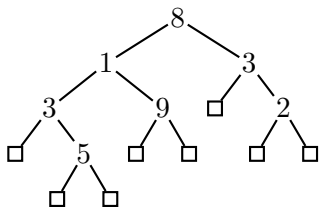
- A **binary tree** is a special case of tree, where each node has either no children or exactly two.



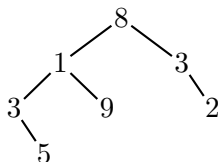
- Nodes are of two kinds: **internal** ($\circ$, $\bullet$ and $\bullet$) or **external** ($\square$).
- The **root** is the only internal node without a parent ($\circ$).
- **Leaves** ($\bullet$) are internal nodes whose children are two external nodes ($\square$).

Binary trees

Internal nodes are usually associated with some kind of information, whilst external nodes are not, like



Extended binary tree



Pruned binary tree

We need to ponder how many shapes a binary tree can take, just like stacks. Each shape as a value constructor and maybe some data.

- The **empty tree** is an external node, without information: `ext()`.
- A **non-empty tree** is an internal node carrying some information x , and which has a left child t_1 and a right child t_2 : `int( $x$ ,  $t_1$ ,  $t_2$ )`.

Binary trees

The previous tree in the following term:

```
int(8, int(1, int(3, ext(),
                int(5, ext(), ext())),
      int(9, ext(), ext())),
  int(3, ext(),
      int(2, ext(), ext()))).
```

We may want to define a function that sums the integers in a binary tree like so:

$$\begin{aligned}\text{add}(\text{ext}()) &\rightarrow 0; \\ \text{add}(\text{int}(x, \text{left}, \text{right})) &\rightarrow x + \text{add } \textit{left} + \text{add } \textit{right}.\end{aligned}$$

The only issue is that we wanted a function that does *not* apply to empty trees (it is partial), but we need the value 0 for the recursion to work.

Optional values

We need another function that checks whether the **whole tree** is empty, and, if, returns a special value that means 'No sum is possible'.

The solution is to use a data type called the **option type**: We simply use `none()` to denote that no value is intended, and 'some v ' to mean that the value v is here.

All we need is to call another function, `add'` that handles the optional (final) value, like so:

$$\begin{aligned}\text{add}'(\text{ext}()) &\rightarrow \text{none}(); \\ \text{add}'(\text{int } node) &\rightarrow \text{some}(\text{add } node).\end{aligned}$$

Notice how we treated `int( $x$ , left, right)` as a tuple, which we named *node*. To make it easier to know that a function is undefined (it is used to denote data structures), we will have its name start with a capital letter, e.g. 'None' and 'Int'.

Tree traversals

Let t be the tree $\text{Int}(x, \text{left}, \text{right})$.

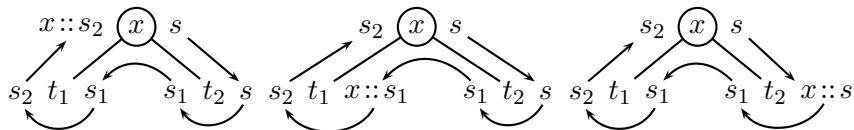
- The **preorder traversal** of t is the list whose first element is x , followed by the elements of the preorder traversal of left , followed by the elements of the preorder traversal of right . The previous tree yields $[8; 1; 3; 5; 9; 3; 2]$.
- The **postorder traversal** of t is the list made of the elements of the postorder traversal of left , followed by the elements of the postorder traversal of right , followed by x . The previous tree yields $[5; 3; 9; 1; 2; 3; 8]$.
- The **inorder traversal** of t is the list made of the elements of the inorder traversal of left , followed by x , followed by the elements of the inorder traversal of right . The previous tree yields $[3; 5; 1; 9; 8; 3; 2]$.

Tree traversals

We want to build the traversals only by pushing values on a stack. We must add an **auxiliary parameter**, originally set to the empty list, on which the contents of the nodes are pushed in the proper order. For preorder:

$$\begin{aligned}\text{pre}(t, s) &\rightarrow \dots \\ \text{pre}'\ t &\rightarrow \text{pre}(t, []).\end{aligned}$$

This extra parameter is the initial value of an accumulator. We should interpret this accumulator when considering the call $\text{pre}(t, s)$. Let us consider the tree $t = \text{Int}(x, t_1, t_2)$ in the following figures:



Preorder (pre)

Inorder (in)

Postorder (post)

Preorder traversal

Each arrow is a step in the traversal and connects different stages of the accumulator: a downwards arrow points to the argument of a recursive call on the corresponding child; an upwards arrow points to the value of the call on the parent. The rules for external nodes are not shown and simply consist in leaving the accumulator invariant.

In the preorder traversal figure, the subtree t_2 is used in the first recursive call $\text{pre}(t_2, s)$ whose value is named s_1 : $\text{pre}(t_2, s) \rightarrow s_1$. Likewise, we have $\text{pre}(t_1, s_1) \rightarrow s_2$. Therefore

$$\text{pre}(t_1, \text{pre}(t_2, s)) \rightarrow s_2.$$

The root is associated with the evaluation $\text{pre}(t, s) \rightarrow x :: s_2$. Consequently,

$$\text{pre}(t, s) \equiv x :: \text{pre}(t_1, \text{pre}(t_2, s)).$$

Three traversals

Finally:

$$\begin{aligned}\text{pre}(\text{Ext}(), s) &\rightarrow s; \\ \text{pre}(\text{Int}(x, t_1, t_2), s) &\rightarrow x :: \text{pre}(t_1, \text{pre}(t_2, s)). \\ \text{pre}' t &\rightarrow \text{pre}(t, []).\end{aligned}$$

Similarly:

$$\begin{aligned}\text{in}(\text{Ext}(), s) &\rightarrow s; \\ \text{in}(\text{Int}(x, t_1, t_2), s) &\rightarrow \text{in}(t_1, x :: \text{in}(t_2, s)). \\ \text{in}' t &\rightarrow \text{in}(t, []).\end{aligned}$$

And

$$\begin{aligned}\text{post}(\text{Ext}(), s) &\rightarrow s; \\ \text{post}(\text{Int}(x, t_1, t_2), s) &\rightarrow \text{post}(t_1, \text{post}(t_2, x :: s)). \\ \text{post}' t &\rightarrow \text{post}(t, []).\end{aligned}$$

Three folds on binary trees

We can abstract the pushing and parameterise pre-, in- and post-order traversals, so they act like a fold on trees:

$$\begin{aligned}\text{fold_pre}(f, \text{Ext}(), s) &\rightarrow s; \\ \text{fold_pre}(f, \text{Int}(x, t_1, t_2), s) &\rightarrow f(x, \text{fold_pre}(f, t_1, \text{fold_pre}(f, t_2, s))). \\ \text{fold_pre}'(f, t) &\rightarrow \text{fold_pre}(f, t, []). \\ \text{pre } t &\rightarrow \text{fold_pre}'((\text{fun } (x, s) \rightarrow x :: s), t).\end{aligned}$$
$$\begin{aligned}\text{fold_in}(f, \text{Ext}(), s) &\rightarrow s; \\ \text{fold_in}(f, \text{Int}(x, t_1, t_2), s) &\rightarrow \text{fold_in}(f, t_1, f(x, \text{fold_in}(f, t_2, s))). \\ \text{fold_in}'(f, t) &\rightarrow \text{fold_in}(f, t, []). \\ \text{in } t &\rightarrow \text{fold_in}'((\text{fun } (x, s) \rightarrow x :: s), t).\end{aligned}$$
$$\begin{aligned}\text{fold_post}(f, \text{Ext}(), s) &\rightarrow s; \\ \text{fold_post}(f, \text{Int}(x, t_1, t_2), s) &\rightarrow \text{fold_post}(f, t_1, \text{fold_post}(f, t_2, f(x, s))). \\ \text{fold_post}'(f, t) &\rightarrow \text{fold_post}(f, t, []). \\ \text{post } t &\rightarrow \text{fold_post}'((\text{fun } (x, s) \rightarrow x :: s), t).\end{aligned}$$

Binary Search Trees

Searching in a binary tree can be costly because, in the worst case, the whole tree must be traversed.

To improve upon the worst case, two scenarios are desirable:

1. the binary tree should be as **balanced** as possible, that is, the nodes should lie as close as possible to the root (for a given notion of distance), and
2. for a given tree, **only the left or the right subtree should be visited**, a decision taken solely by comparing the root and the sought key.

The simplest solution consists in satisfying the second goal, and see later how to also reach the first.

Binary Search Trees

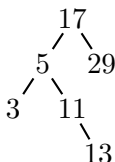
A **binary search tree** is a binary tree where the root (if any) is greater than or equal to the nodes in the left subtree, and lower than the nodes in the right subtree.

In the context of search trees, the nodes are called **keys**, and they need to be **totally ordered** (that is, the fact that any two keys must be comparable).

A consequence is that the inorder traversal of a binary search tree is the list of its keys sorted in non-decreasing order.

Binary Search Trees

Here is an example:



The value for that tree is:

```
t = Int(17, Int(5, Int(3, Ext(), Ext()),  
               Int(11, Ext(),  
                  Int(13, Ext(), Ext()))),  
      Int(29, Ext(), Ext()))
```

The inorder traversal is: $\text{inorder } t \rightarrow [3; 5; 11; 13; 17; 29]$.

Searching

Let consider checking the **membership** of a key k in a binary search tree.

- Any key is absent from the empty tree.
- If the tree is not empty, it has the shape $\text{Int}(x, t_1, t_2)$. We compare k with x : if equal, we found it; if lower, we resume the search in t_1 , else in t_2 .

$$\begin{aligned}\text{mem}(k, \text{Ext}()) &\rightarrow \text{False}(); \\ \text{mem}(k, \text{Int}(x, t_1, t_2)) &\rightarrow \text{mem}(k, t_1), \quad \text{if } k \prec x; \\ \text{mem}(k, \text{Int}(x, t_1, t_2)) &\rightarrow \text{mem}(k, t_2), \quad \text{if } x \prec k; \\ \text{mem}(k, \text{Int}(k, t_1, t_2)) &\rightarrow \text{True}().\end{aligned}$$

Note how the last rule features a **non-linear pattern**. Otherwise, we would have to use the condition $x = k$.

Leaf insertion

Since all unsuccessful searches end at an external node, it is tempting to start the insertion of a unique key by an unsuccessful search and then grow a leaf with the new key at the external node we reached.

If the key is already present in the tree, we decide to leave the tree unchanged.

Let us modify the membership function like so:

$$\begin{aligned} \text{add_leaf}(k, \text{Ext}()) &\rightarrow \text{Int}(k, \text{Ext}(), \text{Ext}()); \\ \text{add_leaf}(k, \text{Int}(x, t_1, t_2)) &\rightarrow \text{Int}(x, \text{add_leaf}(k, t_1), t_2), \quad \text{if } k \prec x; \\ \text{add_leaf}(k, \text{Int}(x, t_1, t_2)) &\rightarrow \text{Int}(x, t_1, \text{add_leaf}(k, t_2)), \quad \text{if } x \prec k; \\ \text{add_leaf}(k, \text{Int}(k, t_1, t_2)) &\rightarrow \text{Int}(k, t_1, t_2). \end{aligned}$$

Root insertion

If recently inserted keys are looked up, the cost is relatively high because these keys are leaves or close to a leaf.

In this scenario, instead of inserting a key as a leaf, it is better to insert it as a root.

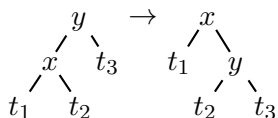
The idea is to perform a leaf insertion and, on the way back to the root (that is to say, after the recursive calls are evaluated, one after the other), we perform rotations to bring the inserted node up to the root.

More precisely, if the node was inserted in a left subtree, then a **right rotation** brings it one level up, otherwise a **left rotation** has the same effect.

The composition of these rotations brings the leaf to the root.

Right rotation

The right rotation can be depicted as



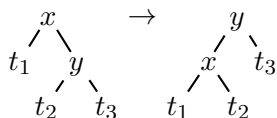
We can see that x goes 'up' and y goes 'down' one level (this is a local transformation).

Let us define 'rotr' (right rotation) like so:

$$\begin{aligned}\text{rotr}(\text{Ext}()) &\rightarrow \text{Ext}(); \\ \text{rotr}(\text{Int}(y, \text{Int}(x, t_1, t_2), t_3)) &\rightarrow \text{Int}(x, t_1, \text{Int}(y, t_2, t_3)).\end{aligned}$$

Left rotation

The left rotation can be depicted as



We can see that x goes 'down' and y goes 'up' one level (this is a local transformation).

Let us define 'rotl' (left rotation) like so:

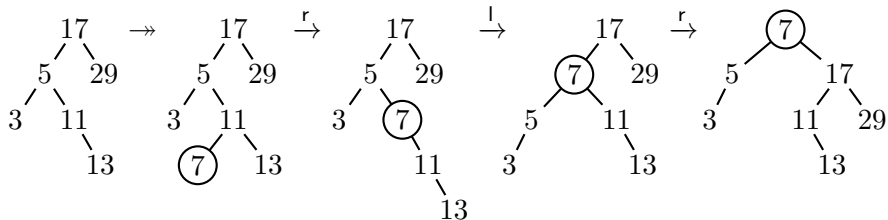
$$\begin{aligned} \text{rotl}(\text{Ext}()) &\rightarrow \text{Ext}(); \\ \text{rotl}(\text{Int}(x, t_1, \text{Int}(y, t_2, t_3))) &\rightarrow \text{Int}(y, \text{Int}(x, t_1, t_2), t_3). \end{aligned}$$

Left and right rotations are the **inverse** of each other:

$$\forall t. \text{rotl}(\text{rotr } t) \equiv \text{rotr}(\text{rotl } t) \equiv t.$$

Root insertion (resumed)

Let us consider an example, where 7 is first added as a leaf, and then a series of rotations, left and right, bring it to the root:



The transitive closure ($\twoheadrightarrow$) captures the preliminary leaf insertion, ($\xrightarrow{r}$) is a right rotation and ($\xrightarrow{l}$) is a left rotation, both applied to the node containing 7.

Note also that the inorder traversal of all those trees remains invariant and a sorted list.

Root insertion

It is now a simple matter to modify the definition of `add_leaf` so it becomes root insertion `add_root` as follows:

$$\begin{aligned} \text{add_root}(k, \text{Ext}()) &\rightarrow \text{Int}(k, \text{Ext}(), \text{Ext}()); \\ \text{add_root}(k, \text{Int}(x, t_1, t_2)) &\rightarrow \text{rotr}(\text{Int}(x, \text{add_root}(k, t_1), t_2)), \quad \text{if } k \prec x; \\ \text{add_root}(k, \text{Int}(x, t_1, t_2)) &\rightarrow \text{rotl}(\text{Int}(x, t_1, \text{add_root}(k, t_2))), \quad \text{if } x \prec k; \\ \text{add_root}(k, \text{Int}(k, t_1, t_2)) &\rightarrow \text{Int}(k, t_1, t_2). \end{aligned}$$

We simply added a right rotation just after performing a left insertion, and a left rotation after a right insertion.

Balancing a binary search tree

Whether we insert a root or a leaf to a binary search tree, we may still obtain **degenerate trees**, that is, trees isomorphic to lists (each internal node has exactly one internal child, except the last).

You may recall that we mentioned page 153 the usefulness of balancing a search tree, so that these extreme cases do not happen.

There exist different criteria to balance a tree, the two most common relying on

1. the **distance** of a node to the root (the path length),
2. the **weight** of a node (the number of nodes in its subtrees).

Unfortunately, discussing and implementing these strategies would lead us too far in this lecture, but you can refer to the literature.

Sorting with Binary Search Trees

We already noticed how insertions leave the inorder traversal of a binary search tree unchanged.

Also the inorder traversal of a binary search tree is the list of its keys sorted in non-decreasing order.

Combining these two facts (which can be made formal claims and proved), we deduce that we can use binary search trees to sort a list of keys chosen from a totally ordered set.

We start by inserting each key in an empty binary search tree.

Next, we collate the inorder traversal of the final tree:

$$\text{sort } l \rightarrow \text{inorder}(\text{fold}(\text{add_leaf}, l, \text{Ext}())).$$

Analysis of algorithms

The mathematical study of the efficiency of algorithms has been pioneered by Knuth, who called it **analysis of algorithms**.

Given a function definition,

1. we define a measure on the arguments, which represents their size;
2. we define a measure on time, which abstracts the wall clock time;
3. we express the abstract time needed to compute calls to that function in terms of the size of its arguments: this is the **cost function**.

The lower the cost, the higher the efficiency.

For example, for sorting objects, called **keys**, by comparing them,

1. the input size is the number of keys,
2. the abstract unit of time is one comparison,
3. and the cost maps the number of keys to the number of comparisons to sort the keys.

Exact cost

Rewrite systems enable a rather natural notion of cost for functional programs: it is the number of rewrites to reach the value of a function call, assuming that the arguments are values.

In other words, the **exact cost** is the number of calls needed to compute the value of a given call.

For stacks, the measure of the size of the input is the number of items it contains, or **length** of the stack.

Exact cost / Appending a stack

For instance, let us recall the appending of a stack:

$$\begin{aligned}\text{cat}([], t) &\xrightarrow{\alpha} t; \\ \text{cat}(x :: s, t) &\xrightarrow{\beta} x :: \text{cat}(s, t).\end{aligned}$$

We observe that t is invariant, so the cost depends only on the size of the first argument.

Let $\mathcal{C}_n^{\text{cat}}$ be the cost of the call $\text{cat}(s, t)$, for any t , where n is the size of s . Each rule yields an equation on their cost:

$$\begin{aligned}\mathcal{C}_0^{\text{cat}} &\stackrel{\alpha}{=} 1, \\ \mathcal{C}_{n+1}^{\text{cat}} &\stackrel{\beta}{=} 1 + \mathcal{C}_n^{\text{cat}}.\end{aligned}$$

Together, they yield $\mathcal{C}_n^{\text{cat}} = n + 1$. Why?

Why?

The trick consists in making the difference of two successive terms in the series and then summing up these differences:

$$C_{n+1}^{\text{cat}} - C_n^{\text{cat}} = 1 \quad \text{where } n \geq 0$$

$$\sum_{n=0}^p (C_{n+1}^{\text{cat}} - C_n^{\text{cat}}) = \sum_{n=0}^p 1 \quad \text{where } p \geq 0$$

$$C_{p+1}^{\text{cat}} - C_0^{\text{cat}} = p + 1$$

Replacing $p + 1$ by n we finally get

$$\begin{aligned} C_n^{\text{cat}} - C_0^{\text{cat}} &= n & \text{where } n > 0 \\ C_n^{\text{cat}} &= n + 1 \end{aligned}$$

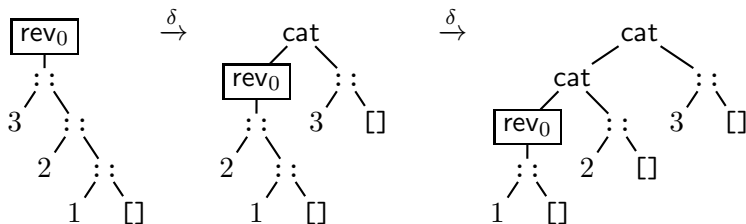
Since that replacing n by 0 in the latter formula gives $C_0^{\text{cat}} = 0 + 1 = 1$, we can have only one formula for the cost: $C_n^{\text{cat}} = n + 1$, where $n \geq 0$. $\square$

Exact cost / Reversal of a list

Let us consider the definition of a function rev_0 reversing a stack:

$$\begin{array}{ll} \text{cat}([], t) \xrightarrow{\alpha} t; & \text{rev}_0 [] \xrightarrow{\gamma} []; \\ \text{cat}(x :: s, t) \xrightarrow{\beta} x :: \text{cat}(s, t). & \text{rev}_0(x :: s) \xrightarrow{\delta} \text{cat}(\text{rev}_0 s, [x]). \end{array}$$

The evaluation of $\text{rev}_0 [3; 2; 1]$ is shown with abstract syntax trees:

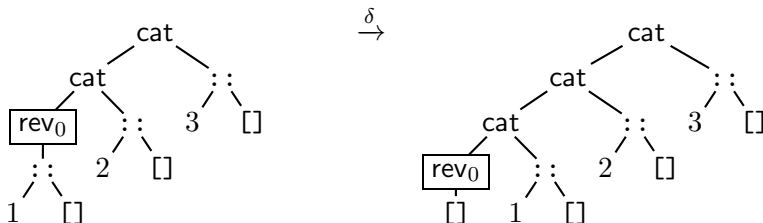


Exact cost / Reversal of a list

Here are the definitions again:

$$\begin{array}{ll} \text{cat}([], t) \xrightarrow{\alpha} t; & \text{rev}_0 [] \xrightarrow{\gamma} []; \\ \text{cat}(x :: s, t) \xrightarrow{\beta} x :: \text{cat}(s, t). & \text{rev}_0(x :: s) \xrightarrow{\delta} \text{cat}(\text{rev}_0 s, [x]). \end{array}$$

Let us resume the evaluation of $\text{rev}_0 [3; 2; 1]$ from where we left:

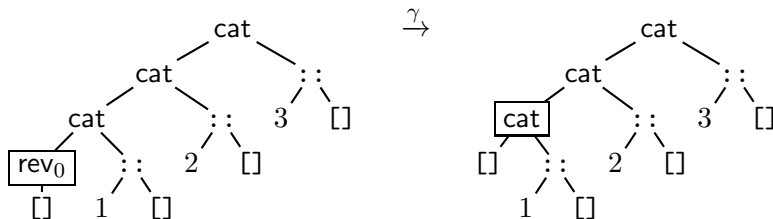


Exact cost / Reversal of a list

Here are the definitions again:

$$\begin{array}{ll} \text{cat}([], t) \xrightarrow{\alpha} t; & \text{rev}_0 [] \xrightarrow{\gamma} []; \\ \text{cat}(x :: s, t) \xrightarrow{\beta} x :: \text{cat}(s, t). & \text{rev}_0(x :: s) \xrightarrow{\delta} \text{cat}(\text{rev}_0 s, [x]). \end{array}$$

Let us resume the evaluation of $\text{rev}_0 [3; 2; 1]$ from where we left:

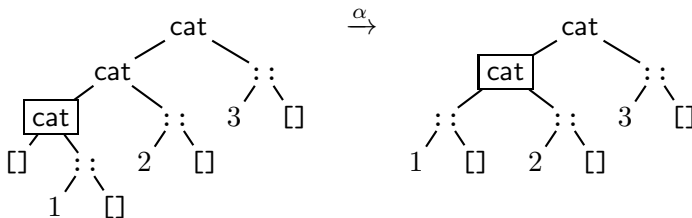


Exact cost / Reversal of a list

Here are the definitions again:

$$\begin{array}{ll} \text{cat}([], t) \xrightarrow{\alpha} t; & \text{rev}_0 [] \xrightarrow{\gamma} []; \\ \text{cat}(x :: s, t) \xrightarrow{\beta} x :: \text{cat}(s, t). & \text{rev}_0(x :: s) \xrightarrow{\delta} \text{cat}(\text{rev}_0 s, [x]). \end{array}$$

Let us resume the evaluation of $\text{rev}_0 [3; 2; 1]$ from where we left:

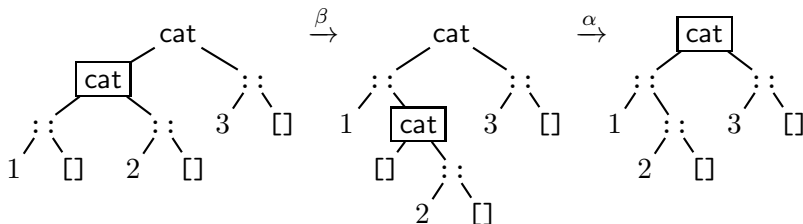


Exact cost / Reversal of a list

Here are the definitions again:

$$\begin{array}{ll} \text{cat}([], t) \xrightarrow{\alpha} t; & \text{rev}_0 [] \xrightarrow{\gamma} []; \\ \text{cat}(x :: s, t) \xrightarrow{\beta} x :: \text{cat}(s, t). & \text{rev}_0(x :: s) \xrightarrow{\delta} \text{cat}(\text{rev}_0 s, [x]). \end{array}$$

Let us resume the evaluation of $\text{rev}_0 [3; 2; 1]$ from where we left:

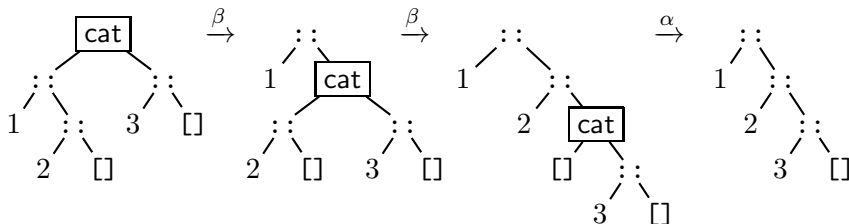


Exact cost / Reversal of a list

Here are the definitions again:

$$\begin{array}{ll} \text{cat}([], t) \xrightarrow{\alpha} t; & \text{rev}_0 [] \xrightarrow{\gamma} []; \\ \text{cat}(x :: s, t) \xrightarrow{\beta} x :: \text{cat}(s, t). & \text{rev}_0(x :: s) \xrightarrow{\delta} \text{cat}(\text{rev}_0 s, [x]). \end{array}$$

Let us resume the evaluation of $\text{rev}_0 [3; 2; 1]$ from where we left:



Exact cost / Reversal of a list

- Here are the definitions again:

$$\begin{array}{ll} \text{cat}([], t) \xrightarrow{\alpha} t; & \text{rev}_0[] \xrightarrow{\gamma} []; \\ \text{cat}(x :: s, t) \xrightarrow{\beta} x :: \text{cat}(s, t). & \text{rev}_0(x :: s) \xrightarrow{\delta} \text{cat}(\text{rev}_0 s, [x]). \end{array}$$

- Let $\mathcal{C}_k^{\text{rev}_0}$ be the cost of evaluating a call $\text{rev}_0(l)$, where the list l has length k .
- Let $\mathcal{C}_k^{\text{cat}}$ be the cost of evaluating a call $\text{cat}(l)$, where the list l has length k .
- The definition of rev_0 directly leads to the recurrence equations

$$\begin{aligned} \mathcal{C}_0^{\text{rev}_0} &\stackrel{\gamma}{=} 1, \\ \mathcal{C}_{k+1}^{\text{rev}_0} &\stackrel{\delta}{=} 1 + \mathcal{C}_k^{\text{rev}_0} + \mathcal{C}_k^{\text{cat}} = \mathcal{C}_k^{\text{rev}_0} + k + 2. \end{aligned}$$

because the length of $\text{rev}_0(s)$ is k if the length of s is k , and we already know $\mathcal{C}_k^{\text{cat}} = k + 1$.

Exact cost / Reversal of a list

We have

$$\sum_{k=0}^{n-1} (C_{k+1}^{\text{rev}_0} - C_k^{\text{rev}_0}) = C_n^{\text{rev}_0} - C_0^{\text{rev}_0} = \sum_{k=0}^{n-1} (k+2) = 2n + \sum_{k=0}^{n-1} k.$$

The remaining sum is a classic of algebra:

$$2 \cdot \sum_{k=0}^{n-1} k = \sum_{k=0}^{n-1} k + \sum_{k=0}^{n-1} k = \sum_{k=0}^{n-1} k + \sum_{k=0}^{n-1} (n-k-1) = n(n-1).$$

Consequently,

$$\sum_{k=0}^{n-1} k = \frac{1}{2}n(n-1).$$

We can finally conclude

$$C_n^{\text{rev}_0} = C_0^{\text{rev}_0} + 2n + \frac{1}{2}n(n-1) = \frac{1}{2}n^2 + \frac{3}{2}n + 1 \sim \frac{1}{2}n^2.$$

Exact cost / Evaluation traces

Another way to reach the result is to induce an **evaluation trace**. A trace is a composition of rewrite rules, noted using the mathematical convention for multiplication.

From the evaluation of $\text{rev}_0[3; 3; 1]$, we infer the trace $\mathcal{T}_n^{\text{rev}_0}$ of the evaluation of $\text{rev}_0(s)$, where n is the length of s :

$$\mathcal{T}_n^{\text{rev}_0} := \delta^n \gamma \alpha (\beta \alpha) \dots (\beta^{n-1} \alpha) = \delta^n \gamma \prod_{k=0}^{n-1} \beta^k \alpha.$$

This reads: ‘Apply δ n times, then γ , then α etc.’

If we note $|\mathcal{T}_n^{\text{rev}_0}|$ the length of $\mathcal{T}_n^{\text{rev}_0}$, that is, the number of rule applications it contains, we expect to have the equations $|x| = 1$, for a rule x , and $|x \cdot y| = |x| + |y|$, for rules x and y .

Exact cost / Evaluation traces

By definition of the cost:

$$\begin{aligned} C_n^{\text{rev}_0} &:= |\mathcal{T}_n^{\text{rev}_0}| = \left| \delta^n \gamma \prod_{k=0}^{n-1} \beta^k \alpha \right| = |\delta^n \gamma| + \sum_{k=0}^{n-1} |\beta^k \alpha| \\ &= (n+1) + \sum_{k=0}^{n-1} (k+1) = (n+1) + \sum_{k=1}^n k = \frac{1}{2}n^2 + \frac{3}{2}n + 1 \sim \frac{1}{2}n^2. \end{aligned}$$

Since we inferred our trace from an example, the cost function we derived may be wrong: we need to prove it by **induction**.

Let $\text{Quad}(n)$ be the property $C_n^{\text{rev}_0} = (n^2 + 3n + 2)/2$. We already know that $\text{Quad}(0)$ is true. Let us suppose $\text{Quad}(n)$ is true for some value of n (induction hypothesis) and let us prove $\text{Quad}(n+1)$.

We know $C_{n+1}^{\text{rev}_0} = C_n^{\text{rev}_0} + n + 2$. The induction hypothesis implies

$$C_{n+1}^{\text{rev}_0} = (n^2 + 3n + 2)/2 + n + 2 = ((n+1)^2 + 3(n+1) + 2)/2,$$

which is $\text{Quad}(n+1)$. Therefore, the induction principle says that the cost we found is always correct. □

Exact cost / Empirical approach

Yet another way to determine the cost of rev_0 consists in first **guessing** that it is quadratic, that is, $C_n^{\text{rev}_0} = an^2 + bn + c$, where a , b and c are unknowns.

Since there are three coefficients, we only need three values of $C_n^{\text{rev}_0}$ to determine them, for instance $n = 0, 1, 2$.

Making some traces, we find $C_0^{\text{rev}_0} = 1$, $C_1^{\text{rev}_0} = 3$ and $C_2^{\text{rev}_0} = 6$, so we solve the linear system

$$C_0^{\text{rev}_0} = c = 1, \quad C_1^{\text{rev}_0} = a + b + c = 3, \quad C_2^{\text{rev}_0} = a \cdot 2^2 + b \cdot 2 + c = 6.$$

Hence $a = 1/2$, $b = 3/2$ and $c = 1$, that is, $C_n^{\text{rev}_0} = (n^2 + 3n + 2)/2$.

We may have been wrong with our guess of a quadratic cost: we need also here to prove it by **induction** exactly as before.

Exact cost / Efficient reversal

We would be remiss not to consider a more efficient (that is, less costly) definition of a reversal function. Here it is:

$$\begin{aligned}\text{rev } s &\xrightarrow{\epsilon} \text{rcat}(s, []). \\ \text{rcat}([], t) &\xrightarrow{\zeta} t; \\ \text{rcat}(x :: s, t) &\xrightarrow{\eta} \text{rcat}(s, x :: t).\end{aligned}$$

(The name 'rcat' stands for *reverse and catenate*.)

The additional parameter introduced by 'rcat' accumulates partial results, thus is called an **accumulator**:

$$\begin{aligned}\text{rev } [3; 2; 1] &\xrightarrow{\epsilon} \text{rcat}([3; 2; 1], []) \\ &\xrightarrow{\eta} \text{rcat}([2; 1], [3]) \\ &\xrightarrow{\eta} \text{rcat}([1], [2; 3]) \\ &\xrightarrow{\eta} \text{rcat}([], [1; 2; 3]) \\ &\xrightarrow{\zeta} [1; 2; 3].\end{aligned}$$

Exact cost / Efficient reversal

To draw the cost of 'rev', it is sufficient to notice that the first argument of 'rcat' strictly decreases at each rewrite, so all evaluation traces have the shape $\epsilon\eta^n\zeta$, so $\mathcal{C}_n^{\text{rev}} = n + 2$. The cost is **linear**, so rev must always be used instead of rev_0 .

The reason for the inefficiency of rev_0 can be seen in the fact that rule δ produces a series of calls to 'cat' following the pattern

$$\text{rev}_0 s \rightarrow \text{cat}(\text{cat}(\dots \text{cat}([], [x_n]), \dots, [x_2]), [x_1]),$$

where $s = [x_1, x_2, \dots, x_n]$. The cost of all these calls to 'cat' is thus

$$1 + 2 + \dots + (n - 1) = \frac{1}{2}n(n - 1).$$

because the cost of $\text{cat}(s, t)$ is $n + 1$. The problem is not calling 'cat', but the fact that the calls are embedded in the most unfavourable configuration.

Extremal costs

When considering sorting programs based on comparisons, the cost varies depending on the algorithm and it also often depends on the original partial ordering of the keys.

Therefore, size does not capture all aspects needed to assess efficiency.

This quite naturally leads to consider bounds on the cost: for a given input size, we seek the configurations of the input that minimise and maximise the cost, respectively called the

- **minimum cost** (cost for the **best case**) and
- **maximum cost** (cost for the **worst case**).

For example, some sorting algorithms have their worst case when the keys are already sorted, others when they are sorted in reverse order, etc.

Average cost

Once we obtain bounds on a cost, the question about the **average** or **mean cost** (also called **expected cost**) arises as well.

It is the arithmetic mean of the costs for all possible inputs of a given size.

Some care is necessary, as there must be a finite number of such inputs.

For instance, to assess the mean cost of sorting algorithms based on comparisons, it is usual to assume

1. that the input is a series of n **distinct keys** and
2. that the sum of the costs is taken over all their **permutations**, thus divided by $n!$, the number of permutations of size n .

Average cost

The uniqueness constraint actually allows the analysis to equivalently, and more simply, consider the permutations of $(1, 2, \dots, n)$.

Some sorting algorithms, like **merge sort** or **insertion sort**, have their average cost **asymptotically equivalent** to their maximum cost.

In other words, for increasingly large numbers of keys, the ratio of the two costs become arbitrarily close to 1.

Some others, like **Hoare's sort**, also known as **quicksort**, have the growth rate of their average cost being of a lower magnitude than the maximum cost, on an asymptotic scale.

Amortised cost

- Sometimes an update is costly because it is delayed by an imbalance in the data structure that calls for an immediate remediation, but this remediation itself may lead to a state such that subsequent operations are faster than if the costly update had not happen.
- Therefore, when considering a series of updates, it may be overly pessimistic to sum the maximum costs of all the operations considered in isolation.
- Instead, **amortised analysis** takes into account the interactions between updates, so a lower maximum bound on the cost is derived.
- Note that this kind of analysis is inherently different from the average case analysis, as its object is the composition of different functions instead of independent calls to the same function on different inputs. **Amortised analysis is a worst case analysis of a sequence of updates.**

A mini functional language

Rewrite systems are a good mathematical and mental model for understanding functional languages.

It is time now to learn an actual functional language, so we can run programs. We are going to start with a mini language that is actually a subset of OCaml, then grow it step by step.

Within rewrite systems, a 'program' is simply a term that is reduced until reaching a stuck term. While this is a good theoretic framework to think about computation in general, it is not conducive to programming in the large, nor understanding the behaviour of programs.

We want programs to be made of different parts that can be reused in different contexts, for example, variables should be defined somewhere and reused at some another points.

Growing a language

This presentation is principled, that is, it relies on a systematic classification of the **syntactic constructs** and their **formal semantics**.

When we reach a certain complexity, we will drop the formal descriptions.

Both theoretical and pragmatic, our approach is complementary to those usually found in books for general audiences.

Let mini-ML be the name of the subset of OCaml we are going to grow.

Appending a stack

The notation we introduced for stacks in the context of rewrite systems actually follows the OCaml convention.

In the context of functional programming, stacks are often called **lists**.

Here are different ways to express the same list in OCaml:

$$1 :: (2 :: 3 :: (4 :: [])) = 1 :: 2 :: 3 :: 4 :: [] = [1; 2; 3; 4].$$

Before starting, let us see how a familiar function defined with a rewrite system looks like in OCaml.

Let us recall the appending of a stack:

$$\begin{aligned}\text{cat}([], t) &\xrightarrow{\alpha} t; \\ \text{cat}(x :: s, t) &\xrightarrow{\beta} x :: \text{cat}(s, t).\end{aligned}$$

Appending a stack

In OCaml, the function name is not repeated (once for each rule).

Instead, it is introduced once by a **binder** called '**let**', like so:

let cat = **function** ...

The complete definition in OCaml is:

let rec cat = **function**

[], $t \rightarrow t$

| $x :: s, t \rightarrow x :: \text{cat}(s, t)$

Note the addition of the keyword **rec**: this is meant so the call $\text{cat}(s, t)$ indeed refers to the function being defined.

With rewrite systems, recursion is not an issue: any call matching a pattern can be rewritten.

Note also how the two rules are introduced by the keyword **function** and are separated by a vertical line.

Sentences

An OCaml program is a sequence of sentences. A **sentence**, or **global definition**, is defined by the following cases, where e denotes an expression, x is a variable, and '**let**' and '**rec**' are keywords:

- *global definition* **let** $x = e$
- *global recursive definition* **let rec** $x = e$

Global definitions can be optionally followed by two semicolons: these are necessary when inputting the sentences in the toplevel loop (prompted after running the command `ocaml` from a Unix shell).

Note that the keyword '**rec**' is necessary to enable recursion. (We will see later how that works precisely.)

For e to be an expression means that e denotes a part of a sentence which is classified as an expression according to its **syntax**.

Metavariables

- We say that e is a **metavariable** because, being a name, it is a variable, but that variable does not belong to the language being described (the **object language**, here, OCaml) and, instead, exists only in the descriptive language (the **metalanguage**).
- In the following

let $x = e$

the metavariable x (mind the italics) denotes an infinite set of OCaml variables and we must not confuse it with the OCaml variable x (no italics). Similarly, the metavariable e denotes an infinity of expressions, as in the following:

let $e = 3$

let $x = e$

let $a = x + e$

Expressions

- An expression e is **recursively defined by** the following **cases**:
 - *variable* $\text{my_var, } x, y, z, \text{ etc.}$
 - *unit or integer constant* $() \text{ or } 0 \text{ or } 1 \text{ or } 2, \text{ etc.}$
 - *function (or abstraction)* **fun** $x \rightarrow e$
 - *call (or application)* $e_1 \ e_2$
 - *arithmetic operator* $(+) \ (-) \ (/) \ (*)$
 - *arithmetic operation* $e_1 + e_2 \text{ or } e_1 - e_2, \text{ etc.}$
 - *parenthesis* (e)
 - *local definition* **let** $x = e_1 \text{ in } e_2$
- Note how the keywords '**rec**' and '**function**', as well as the definition of multiple cases (as seen in the definition of 'cat' earlier) is not found above: this is because we want to focus on other aspects of OCaml first, like higher-order functions.
- Instead, we have '**fun**', which enables only one case made of a variable (more generally, irrefutable patterns).

Examples of expressions

- Note that what we print nicely as ' $\rightarrow$ ' is actually written ' $->$ ' in the source code.
- Here are examples of OCaml sentences making up a program, step by step.
- First, a numerical constant:

let $x = 0$

- Note that constants can be defined by means of a rewrite system as functions taking the empty tuple:

$x() \rightarrow 0$

- The next sentence is

let $id = \text{fun } x \rightarrow x$

- This is equivalent to the rewrite system

$id(x) \rightarrow x$

Examples of expressions

- Next we have another constant being defined by binding a variable to the value of a call:

let $y = 2$ **in** $\text{id } y$

This is equivalent to the call $\text{id}(2)$.

- Now

let $x = (\text{fun } x \rightarrow \text{fun } y \rightarrow x + y)$ $1\ 2$

is equivalent to

let $x = 1 + 2$

(We will see why later.)

- Finally

let $z = x + 1$

is obvious.

Functions

- Variables must start with a lowercase character.
- The arrow is right-associative, so the expression

$$\mathbf{fun} \ x_1 \rightarrow \mathbf{fun} \ x_2 \rightarrow \dots \rightarrow \mathbf{fun} \ x_n \rightarrow e$$

is equivalent to

$$\mathbf{fun} \ x_1 \rightarrow (\mathbf{fun} \ x_2 \rightarrow (\dots \rightarrow (\mathbf{fun} \ x_n \rightarrow e)) \dots),$$

or, even shorter:

$$\mathbf{fun} \ x_1 \ x_2 \dots x_n \rightarrow e.$$

- Function calls are left-associative, so

$$e_1 \ e_2 \ e_3 \dots e_n \triangleq (((\dots (e_1 \ e_2) \ e_3) \dots) \ e_n).$$

- Function calls have higher priority than operator calls, for example,

$$f \ 3 + 4 = (f \ 3) + 4 = f(3) + 4.$$

Function calls

Operator calls have higher syntactic priority than abstractions, for instance,

fun $x \rightarrow x + y$ is the same as **fun** $x \rightarrow (x + y)$.

Note the difference between $f \times y$ and $f(x, y)$.

The former is to be understood as $(f(x))(y)$ which means that there are *two* calls:

1. First, the call $f(x)$ is evaluated. Let us assume that its value is g .
2. Second, the call $g(y)$ is evaluated.

In other words, the first case is equivalent to

let $g = f \times$ **in** $g y$

Abstract syntax

A graphical representation of programs (here, expressions) as trees proves very handy when studying their semantics (meaning):

Flat view	Tree view
x	x
fun $x \rightarrow e$	fun $\swarrow \searrow$ $x \quad e$
$e_1 \ e_2$	$\$$ $\swarrow \searrow$ $e_1 \ e_2$

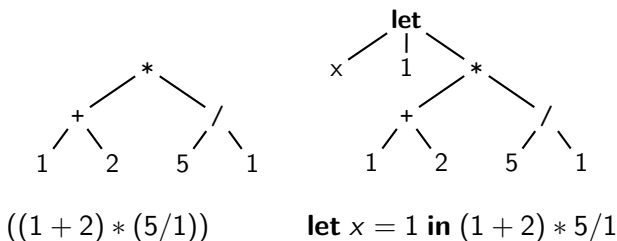
Note that we needed some symbol for the root of the application tree ($\$$). Traditionally, that symbol is ($@$), but OCaml uses it already for something else.

When trees represent programs, they are called **abstract syntax trees** (AST).

Abstract syntax

Intuitively, the method for constructing abstract syntax trees consists in fully parenthesising the expression. Each parenthesis corresponds to a subexpression and each subexpression corresponds to a subtree.

The tree is built from the root, down to the leaves, by traversing subexpressions from the outermost to the innermost, that is, the most embedded ones. For instance, consider

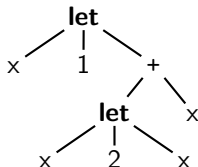


Abstract syntax

Another example:

let $x = 1$ **in** ((**let** $x = 2$ **in** x) + x)

corresponds to the abstract syntax tree



This example features two local definitions of 'x': is one the redefinition of the other? Is one hiding the other?

There are also two uses of 'x': which use corresponds to which definition?

Bindings, environments, scope

To answer these questions, we must first understand

- what a **binding** is,
- what the **scope** of a binding is, and
- what an **environment** is.

The expression we called **local definition** earlier:

$$\mathbf{let\ } x = e_1 \mathbf{\ in\ } e_2$$

is evaluated as follows:

1. The expression e_1 is evaluated,
2. assuming it has a value v_1 , then v_1 is **bound** to the variable x ,
3. the expression e_2 is evaluated under the assumption that x is bound to v_1 .

Bindings, environments, scope

A local definition

let $x = e_1$ **in** e_2

associates the value v_1 of expression e_1 to the variable denoted by x , making up a **binding** noted $x \mapsto v_1$, and then proceeds to evaluate e_2 with this binding available.

The binding $x \mapsto v_1$ is not usable outside the evaluation of e_2 . We say that the **scope** of the definition is made of the subtrees of the abstract syntax tree corresponding to e_2 where the binding of that definition is usable ('visible').

A mapping from variables to values is called an **environment**. Let us note $\rho_\emptyset$ the empty environment. We write

$$\rho[x \mapsto v]$$

to mean that the binding $x \mapsto v$ was added to the environment ρ .

Bindings, environments, scope

Environments are functions and are updated as follows:

$$\rho[x \mapsto v](y) \triangleq \begin{cases} v & \text{if } x = y, \\ \rho(y) & \text{otherwise.} \end{cases} \quad (1)$$

with $\forall x. \rho_{\emptyset}(x) = \perp$ and $\perp$ is interpreted as an **error**. Errors are special values. Consider again the program

```
let x = 0 in  
let id = fun x  $\rightarrow$  x in  
let y = id x in  
let x = (fun x y  $\rightarrow$  x + y) 1 2  
in x + 1
```

The variable x in the expression $x+1$ denotes the value of the expression bound to the variable x in the previous line, not the first.

Bindings, environments, scope

Bindings are ordered in the environment **by the order of their definitions**. Let us follow the evolution of the environment.

1. The environment is initially empty: $\rho_{\emptyset}$;
2. **let** $x = 0$ **in** ...
yields $\rho_{\emptyset}[x \mapsto 0]$
3. **let** $id = \text{fun } x \rightarrow x$ **in** ...
yields $\rho_{\emptyset}[x \mapsto 0][id \mapsto \text{fun } x \rightarrow x]$;
4. **let** $y = id\ x$ **in** ...
yields $\rho_{\emptyset}[x \mapsto 0][id \mapsto \text{fun } x \rightarrow x][y \mapsto 0]$;
5. **let** $x = \dots$
yields $\rho_{\emptyset}[x \mapsto 0][id \mapsto \text{fun } x \rightarrow x][y \mapsto 0][x \mapsto 3]$.
6. The expression $x+1$ is evaluated in $3 + 1 = 4$ because $x \mapsto 3$ (the last binding) **hides** $x \mapsto 0$ (the first binding).
7. In other words, the first definition of x was **out of scope** in $x+1$.

Free variables

- If a variable x is not bound to a value in an environment ρ , we say that x is **free** in ρ .
- By definition, a variable is **free** in an expression if it is not bound by a **'let'** or a **'fun'**.
- We can understand this concept on a graphic representation of the abstract syntax tree.
- From each variable occurrence, we move up, towards the root.
- If we encounter a **'let'** that binds that variable (in the left child), we create an oriented edge from the variable to that **'let'**, and we stop: the variable is bound.
- Otherwise, if we reach the root (no such **'let'** has been found), then the variable is free.

Free variables

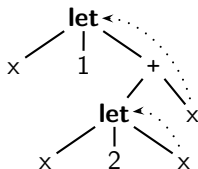


Figure: `let x = 1 in ((let x = 2 in x) + x)`

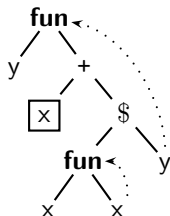


Figure: `fun y -> x + (fun x -> x) y`

Free variables

A similar situation arises with functions: '**fun**' is a binder, just like '**let**'.

In **fun** $x \rightarrow e$, the variable x (called a **parameter**) may shadow (hide) another variable x defined on the path to the root.

See previous page for two examples.

We note $\mathcal{F}(e)$ the set of the variables free in e .

Free variables

It is easy to formally define $\mathcal{F}$ by case and recursively:

$$\mathcal{F}(x) = \{x\}, \quad \text{where } x \text{ is a variable}$$

$$\mathcal{F}(c) = \emptyset, \quad \text{where } c \text{ is a constant}$$

$$\mathcal{F}(\text{fun } x \rightarrow e) = \mathcal{F}(e) \setminus \{x\}$$

$$\mathcal{F}(e_1 \ e_2) = \mathcal{F}(e_1) \cup \mathcal{F}(e_2)$$

$$\mathcal{F}(\text{let } x = e_1 \text{ in } e_2) = \mathcal{F}(e_1) \cup \mathcal{F}(e_2) \setminus \{x\}$$

$$\mathcal{F}(e_1 + e_2) = \mathcal{F}(e_1) \cup \mathcal{F}(e_2)$$

Now, instead of drawing abstract syntax trees, let us use these equations to compute the free variables of our previous examples.

Free variables

$$\begin{aligned}\mathcal{F}(\text{let } x = 1 \text{ in } (\text{let } x = 2 \text{ in } x) + x) \\&= \mathcal{F}(1) \cup \mathcal{F}((\text{let } x = 2 \text{ in } x) + x) \setminus \{x\} \\&= \emptyset \cup (\mathcal{F}(\text{let } x = 2 \text{ in } x) \cup \mathcal{F}(x)) \setminus \{x\} \\&= (\mathcal{F}(2) \cup \mathcal{F}(x) \setminus \{x\} \cup \mathcal{F}(x)) \setminus \{x\} \\&= \mathcal{F}(x) \setminus \{x\} \\&= \emptyset.\end{aligned}$$

$$\begin{aligned}\mathcal{F}(\text{fun } y \rightarrow x + (\text{fun } x \rightarrow x) y) \\&= \mathcal{F}(x + (\text{fun } x \rightarrow x) y) \setminus \{y\} \\&= (\mathcal{F}(x) \cup \mathcal{F}(\text{fun } x \rightarrow x) y) \setminus \{y\} \\&= (\{x\} \cup \mathcal{F}(\text{fun } x \rightarrow x) \cup \mathcal{F}(y)) \setminus \{y\} \\&= (\{x\} \cup \mathcal{F}(x) \setminus \{x\} \cup \{y\}) \setminus \{y\} \\&= \{x, y\} \setminus \{y\} \\&= \{x\}.\end{aligned}$$

Closed expressions

- A **closed** expression is an expression without free variables.
- That is, an expression e is closed if, and only if, $\mathcal{F}(e) = \emptyset$.
- By extension, we say that e is **closed in an environment** ρ if $\mathcal{F}(e) \subseteq \text{dom}(\rho)$ (the free variables are bound in ρ).
- Only a closed expression can be evaluated (executed). That is why the first static analysis performed by compilers consists in determining the variables which are free in expressions: if the program is not closed, it is rejected.
- In the case of

fun $y \rightarrow x + (\text{fun } x \rightarrow x) y$,

the OCaml compiler prints out

Unbound value x

and stops. It is useful that this open expression be rejected at compile-time and does not cause a run-time error.

Evaluation

Evaluation is a partial function from expressions to values. An **interpreter** is a program implementing an evaluation. The partiality models the fact that an evaluation may not terminate or end in an error.

With rewrite systems, values are defined as stuck expressions, that is, expressions that cannot be rewritten further.

Here, since we want to define expressions (terms) whose value we can reuse somewhere else (in another term), we need the notion of **environment**.

That implies that a formal account of evaluation (the equivalent for programs of rewriting terms) needs to take into account environments.

Semantic values

This goes even further because we want Mini-ML to be **higher-order**, that is, we want functions to be values and this begets the question of the meaning of the free variables in the function bodies: after passing around a function as argument to other functions, what are the values of those variables when the threaded function is finally applied?

The most common answer is **lexical (or static) scoping**, whereby the free variables in a body remain bound to their values at the point the function was defined.

This means that values corresponding to functions have to carry around the environment at the point of their definition.

One common way to achieve this is to have a notion of **semantic value**, which differs from **syntactic value**. For example, the mathematical integers are part of the semantic values of Mini-ML, but not of syntactical values.

Semantic values

- The **semantic values** are defined recursively by the following cases:
 - *constant* $()$, 0, 1, 2, etc.
 - *closure* $\langle \mathbf{fun} \ x \rightarrow e; \rho \rangle$, with environment ρ .
 - *n-tuple* $v_1, \dots, v_n$, with v_i values.
- The closure environment ρ is the one at the location where the function **fun** $x \rightarrow e$ occurs, and the function is closed in that environment (this is called **static scoping**, or **lexical scoping**).
- The evaluation of an expression must take place in the context of an environment, where all free variables must be bound (in other words, the expression must be closed in that environment).
- For example, let ρ be the environment before the sentence

let $x = 1$ **in** x

Then the evaluation of the *second* ' x ' must be done in the environment $\rho[x \mapsto 1]$, where ' x ' is the *first* one.

Evaluation

Let us proceed to define a semantics for our mini-ML, by associating to each expression e its value v , assuming the environment ρ :

- x Look up the **last** value bound to x in ρ .
- **fun** $x \rightarrow e$ The value is the closure $\langle \text{fun } x \rightarrow e; \rho \rangle$.
- $e_1 + e_2$, etc. Evaluate e_1 and e_2 in ρ , then add, etc.
- $()$ or 0 or 1 or 2, etc. The value is $()$ or 0 or 1, etc.
- (e) Evaluate e into v in ρ .
- **let** $x = e_1$ **in** e_2 Evaluate e_1 into v_1 in ρ ;
evaluate e_2 into v in $\rho[x \mapsto v_1]$.
- $e_1 \ e_2$ Evaluate e_1 and e_2 into v_1 and v_2 in ρ
(order left unspecified);
 v_1 must be the closure $\langle \text{fun } x \rightarrow e; \rho_1 \rangle$;
evaluate e in $\rho_1[x \mapsto v_2]$;
the value is v .

Example of evaluation

1. The environment is initially empty. Let us evaluate

```
let x = 0 in  
let id = fun x  $\rightarrow$  x in  
let y = id x in  
let x = (fun x y  $\rightarrow$  x + y) 1 2  
in x + 1
```

2. The evaluation of

```
let x = 0 in ...
```

starts with the evaluation of 0, which, obviously, yields 0.

3. We create the binding $x \mapsto 0$, with which we extend $\rho_\emptyset$, that is, $\rho_\emptyset[x \mapsto 0]$.
4. We evaluate the elided subexpression within this new environment.

Example of evaluation

- The evaluation of

let id = **fun** x $\rightarrow$ x **in** ...

is performed within the environment $\rho_{\emptyset}[x \mapsto 0]$.

- The value of 'id' is then the closure $\langle \text{fun } x \rightarrow x; \rho_{\emptyset}[x \mapsto 0] \rangle$.
- We extend the current environment and evaluate the elided subexpression within it.
- The evaluation of

let y = id x **in** ...

is done within the environment

$\rho = \rho_{\emptyset}[x \mapsto 0][\text{id} \mapsto \langle \text{fun } x \rightarrow x; \rho_{\emptyset}[x \mapsto 0] \rangle]$.

- We evaluate first (id x) within ρ .
- In order to do so, we evaluate 'id' and 'x' separately.

Example of evaluation

- These are both variables, thus we look them up in the environment to retrieve their last corresponding binding:
 $\rho(\text{id}) = \langle \text{fun } x \rightarrow x; \rho_{\emptyset}[x \mapsto 0] \rangle$ and $\rho(x) = 0$.
- We need to evaluate the body in the environment $\rho' = \rho_{\emptyset}[x \mapsto 0][x \mapsto 0]$, which yields $\rho'(x) = 0$.
- We create the binding $y \mapsto 0$, we add it to the current environment and we evaluate the elided subexpression with the new environment.
- Let us stop here and move to expressing the above semantics in a formal manner.

Formal semantics

If we note $\llbracket e \rrbracket \rho$ the value obtained by evaluating the expression e in the environment ρ , then the evaluation of mini-ML is:

$$\llbracket \bar{n} \rrbracket \rho = n, \text{ with } \bar{n} \text{ an OCaml integer and } n \in \mathbb{N}$$

$$\llbracket e_1 + e_2 \rrbracket \rho = \llbracket e_1 \rrbracket \rho + \llbracket e_2 \rrbracket \rho \in \mathbb{N}$$

$$\llbracket (e) \rrbracket \rho = \llbracket e \rrbracket \rho$$

$$\llbracket x \rrbracket \rho = \rho(x)$$

$$\llbracket \mathbf{fun} \ x \rightarrow e \rrbracket \rho = \langle \mathbf{fun} \ x \rightarrow e; \rho \rangle$$

$$\begin{aligned} \llbracket e_1 \ e_2 \rrbracket \rho &= \llbracket e_3 \rrbracket \rho_3 [x \mapsto \llbracket e_2 \rrbracket \rho], \\ &\text{where } \llbracket e_1 \rrbracket \rho = \langle \mathbf{fun} \ x \rightarrow e_3; \rho_3 \rangle \end{aligned}$$

$$\llbracket \mathbf{let} \ x = e_1 \ \mathbf{in} \ e_2 \rrbracket \rho = \llbracket (\mathbf{fun} \ x \rightarrow e_2) \ e_1 \rrbracket \rho.$$

Evaluation means applying the equations from left to right.

Partial applications

- We can easily work out that

$$\llbracket \text{let } x = e_1 \text{ in } e_2 \rrbracket \rho = \llbracket e_2 \rrbracket \rho[x \mapsto \llbracket e_1 \rrbracket \rho].$$

- Since closures are values, they can be the value of a function call:

```
let add = fun x y → x + y in
let incr = add 1
in incr 5
```

- The call (add 1) is a **partial application**, as opposed to a **complete application** like (add 1 5). A partial application, by definition, always evaluates in a closure.
- Operations are functions denoted by a symbol used in infix position (between the operands). As such, they can be partially evaluated:

```
let incr = (+) 1 in incr 5
```

where the parentheses in (+) mean that the operator has to be considered in **prefix** position.

Auto-application and termination

Our mini-ML is **Turing-complete**, just as OCaml is. For instance, we can write the following non-terminating program:

let $\Omega = \mathbf{fun} \ x \rightarrow x \ x \ \mathbf{in} \ \Omega \ \Omega$

The function Ω is an **auto-applicative function**. It enables the following non-terminating evaluation:

$$\begin{aligned} \llbracket \mathbf{let} \ \Omega = \mathbf{fun} \ x \rightarrow x \ x \ \mathbf{in} \ \Omega \ \Omega \rrbracket \rho &= \llbracket \Omega \ \Omega \rrbracket \rho [\Omega \mapsto \llbracket \mathbf{fun} \ x \rightarrow x \ x \rrbracket \rho] \\ &= \llbracket x \ x \rrbracket \rho [x \mapsto \langle \mathbf{fun} \ x \rightarrow x \ x; \rho \rangle] \\ &= \textit{idem}. \end{aligned}$$

Fixed points

- To demonstrate the expressive power of our mini-ML, let us see how we can simulate recursive functions by means of the Ω function:

$$\text{let } \Omega = \text{fun } x \rightarrow x \ x$$

- Let us define a function `fix`, called the **Y combinator** in λ -calculus:

$$\text{let fix} = \text{fun } g \rightarrow \Omega \ (\text{fun } h \ x \rightarrow g \ (h \ h) \ x)$$

- For any (terminating) function f and any function x ,

$$\llbracket f \ (\text{fix } f) \ x \rrbracket \rho = \llbracket \text{fix } f \ x \rrbracket \rho.$$

- In other words, we have

$$f \ (\text{fix } f) \ x \equiv \text{fix } f \ x.$$

- The fixed point x of a (continuous) function g satisfies $g(x) = x$.
- Therefore, the fixed point of 'f' (if it exists) is the value of 'fix f'.

Fixed points

Just as with **let in**, we can simplify the semantics of ‘fix’ a bit:

$$\llbracket \text{fix } e \rrbracket \rho = \llbracket e_1 \rrbracket \rho_1 [x \mapsto \llbracket \text{fix } (\mathbf{fun } x \rightarrow e_1) \rrbracket \rho],$$

where $\llbracket e \rrbracket \rho = \langle \mathbf{fun } x \rightarrow e_1; \rho_1 \rangle$.

There is a choice:

- either we define ‘fix’ in the functional language (as a built-in constant), or
- at the meta-language (the mathematical logic we are using to define the semantics).

Fixed points

Let us consider the following definitions:

```
let pre_fact = fun f n  $\rightarrow$  if n = 1 then 1 else n * f(n - 1)  
let fact = fix pre_fact
```

We see that 'fact' is the fixed point of 'pre_fact':

$$\begin{aligned}\llbracket \text{fact} \rrbracket \rho &= \llbracket \text{fix pre_fact} \rrbracket \rho \\ &= \llbracket \text{pre_fact (fix pre_fact)} \rrbracket \rho \\ &= \llbracket \text{pre_fact fact} \rrbracket \rho \\ &= \llbracket \text{fun } n \rightarrow \text{if } n = 1 \text{ then } 1 \text{ else } n * \text{fact}(n - 1) \rrbracket \rho\end{aligned}$$

A non-recursive definition of the factorial.

Fixed points in C

```
#include<stdio.h>
#include<stdlib.h>

typedef int (*fp)();

int fact(fp f, int n) {
    return n? n * ((int (*)(fp,int))f)(f,n-1) : 1; }

int read(int dec, char arg[]) {
    return ('0' <= *arg && *arg <= '9')?
        read(10*dec + *arg - '0', arg+1) : dec; }

int main(int argc, char** argv) {
    if (argc == 2)
        printf("%u\n", fact(&fact, read(0, argv[1])));
    else printf("Only one integer allowed.\n");
    return 0; }
```

Local recursive definitions

- The function 'fact' coincides pointwise with the factorial function.
- Let us extend mini-ML with a native recursive binder, using the 'fix' function:

$$\llbracket \text{let rec } x = e_1 \text{ in } e_2 \rrbracket \rho = \llbracket \text{let } x = \text{fix } (\text{fun } x \rightarrow e_1) \text{ in } e_2 \rrbracket \rho.$$

- In practice, recursive definitions are given a built-in semantics, as opposed to being defined in terms of a special combinator like 'fix'.

Extensions

- Let us extend our mini-ML with the following expressions.
 - *Boolean constants* **true** or **false**
 - *Boolean operators* (**&&**) or (**||**) or **not**
 - *n-tuple* $e_1, \dots, e_n$
 - *conditional* **if** e_0 **then** e_1 **else** e_2
 - *local recursive binding* **let rec** $f = e_1$ **in** e_2
- Note that parentheses are recommended around tuples, but are not mandatory.
- Let us distinguish the variables occurring after **let** and **fun**, because they are **irrefutable patterns**, noted $\overline{p}$:
 - *function* **fun** $\overline{p} \rightarrow e$
 - *local definition* **let** [**rec**] $\overline{p} = e_1$ **in** e_2

Irrefutable patterns

- An irrefutable pattern $\bar{p}$ is defined recursively by the following cases:
 - *variable* f, g, h, x, y, z , etc.
 - *unit* $()$
 - *n-tuple* $\bar{p}_1, \dots, \bar{p}_n$
 - *parenthesis* $(\bar{p})$
 - *underscore* $_$
- Note that, from the syntactic point of view, irrefutable patterns are special cases of expressions, except the underscore, which is a special case avoiding the creation of a binding.
- The sentence e is equivalent to **let** $_ = e$.

Extensions

- The comma takes priority over the arrow:

fun $x \rightarrow x, y$ is the same as **fun** $x \rightarrow (x, y)$.

- In order to alleviate the notation **fun** $\bar{p}_1 \rightarrow \dots \rightarrow$ **fun** $\bar{p}_n \rightarrow e$, we define the equivalent expressions:
 - **let** [**rec**] $f =$ **fun** $\bar{p}_1 \bar{p}_2 \dots \bar{p}_n \rightarrow e$ (new expression)
 - **let** [**rec**] $f \bar{p}_1 \bar{p}_2 \dots \bar{p}_n = e$ (new sentence)
- For example, we would write

let mult = **fun** $x\ y \rightarrow x * y$ **in**

let eq $x\ y = (x = y)$ **in**

let rec fact $n =$ **if** eq $n\ 0$ **then** 1 **else** mult n (fact($n - 1$))

in fact 9

- Note the two different meanings of (=): one is definitional, the other is operational (a Boolean operator).

Parallel definitions

- We also extend the syntax to alleviate certain expressions. By definition,

let $\bar{p}_1 = e_1$ **and** $\bar{p}_2 = e_2 \dots$ **and** $\bar{p}_n = e_n$ **in** e

is equivalent to

let $\bar{p}_1, \dots, \bar{p}_n = e_1, \dots, e_n$ **in** e

but the latter requires more memory (for the tuples in the heap).

- These are called **parallel definitions** because they occur within the same environment and could be conceived as happening in parallel.
- The two following sentences are equivalent:

let $x = 1$ **and** $y = 2$ **in** $x + y$

let $x, y = 1, 2$ **in** $x + y$

Mutually recursive definitions

- Parallel definitions are useful for **mutually recursive definitions**:
 - **let rec** $\bar{p}_1 = e_1$ **and** $\bar{p}_2 = e_2 \dots$ **and** $\bar{p}_n = e_n$ **in** e (expression)
 - **let rec** $\bar{p}_1 = e_1$ **and** $\bar{p}_2 = e_2 \dots$ **and** $\bar{p}_n = e_n$ (sentence)
- A silly example:

let rec even $n =$ **if** $2 * (n/2) = n$ **then true else** odd($n - 1$)
and odd $n =$ **if** $2 * (n/2) = n$ **then false else** even($n - 1$)

where $(/)$ is the Euclidean division.

Static typing and inference

- The aim of **static typing** is to detect and reject at compile-time some kinds of programming errors which would otherwise result into an error at run-time, like $(1\ 2)$ or $(\text{fun } x \rightarrow x) + 1$.
- With this aim, a **type** is assigned by the compiler or the programmer to each subexpression of an expression (in particular, the whole program).
- For example, 'int' for an arithmetic expression, or $\text{int} \rightarrow \text{int}$ for a function from the integers to the integers.
- The **coherence** of these types is checked by the OCaml compiler, that is, if we think of a type as an assumption, no assumption must contradict another one.

Static typing and inference

- It is undecidable to determine all possible run-time errors for all programs before running them.
- **Type systems**, on the other hand, are often **decidable**, because it is considered a good design to ensure that the compiler terminates on all input programs.
- Therefore, it is impossible for the compiler to reject only those programs that would 'go wrong' at run-time.
- In other words, all type systems reject innocent programs, and that is why the quest for better type systems, with better compromises between unfair rejection and guaranteed safety, will never end.

Types

- **Types are terms** associated to expressions, either by the programmer or the compiler, and they capture some **invariant properties** which can be automatically composed and checked.
- A type t is a term defined recursively by the following cases:
 - *simple* char, bool, int, string, float, unit
 - *Cartesian product* $t_1 \times \dots \times t_n$
 - *functional* $t_1 \rightarrow t_2$
 - *parenthesis* (t)
 - *free variable* α, β, γ , etc.
 - *parameterised* α list
- Until now, we have not encountered values of the type float, char or string, but they are fairly obvious.
- We write $\times$, α , β , etc. what is written $*$, 'a', 'b', etc. in the source code.

Type expressions

- The Cartesian product takes multiple arguments, instead of being binary like in mathematics, and $(\times)$ is not associative in OCaml:

$$t_1 \times t_2 \times t_3 \neq (t_1 \times t_2) \times t_3 \neq t_1 \times (t_2 \times t_3).$$

- The arrow is used in types too, where it is also right-associative:

$$t_1 \rightarrow t_2 \rightarrow \dots \rightarrow t_{n-1} \rightarrow t_n$$

is equivalent to

$$t_1 \rightarrow (t_2 \rightarrow (\dots (t_{n-1} \rightarrow t_n)) \dots).$$

- The Cartesian product has priority over the arrow:

$$t_1 \times t_2 \rightarrow t_3$$

is equivalent to

$$(t_1 \times t_2) \rightarrow t_3.$$

Type expressions

- The OCaml compiler computes a type for each expression: this is called **static type inference**.
- For the simple constants, we have

Type	Values	Some functions
unit	()	
bool	true false	&& not
int	1 2 max_int	+ - * /
float	1.0 2. 1e4	+. -. *. /. cos
char	'a' '\n' '\097'	Char.code Char.chr
string	"a\tb\010c\n",	(^) s.[i]

- Operations on floating point numbers are written differently from their equivalent counterparts on integers.

Type definitions

- We further extend the syntax of the sentences to allow the binding of a type to a name, or **type definition**:
 - *type definition* **type** [**nonrec**] $q = t$
 - *parallel type definition* **type** [**nonrec**] $q_1 = t_1$ [**and** $q_2 = t_2 \dots$]
 - *type variable* qIn particular:
 - *type aliasing* **type** $q_0 = q_1$
- By default, type definitions can be recursive (we will see an example later). If not, use the keyword **nonrec**.
- We can now write, for instance, the following:

```
type abscissa = float
type ordinate = float
type point = abscissa × ordinate
type origin = point
```

Type inference

- Tuples contain data of the same type and their arity is part of their type.
- For instance, the pair (1,2) has type $\text{int} \times \text{int}$, and the triplet (1,2,3) has type $\text{int} \times \text{int} \times \text{int}$, thus are incompatible.
- Consider also the following session at the OCaml toplevel:

```
# let middle x y = (x+y)/2;;
```

```
val middle : int → int → int
```

```
# let middle (x,y) = (x+y)/2;;
```

```
val middle : int × int → int
```

- In the above, the OCaml compiler figures out in both cases that, since we compute $(x+y)$, that implies that x and y must have type `int`.

Polymorphism

- When a variable is not constrained by its uses, like x in **fun** $x \rightarrow x$, it is assigned the most general type, that is, a **type variable**:
let $id = \text{fun } x \rightarrow x;;$
val $id : \alpha \rightarrow \alpha = \langle \text{fun} \rangle$
- This means that the above identity function can be applied to any kind of value, and that the type of the call is that of the argument.
- This property of a function to accept values of different types is called **parametric polymorphism**. Consider more examples:

fun $(x,y,z) \rightarrow x;;$

- : $\alpha \times \beta \times \gamma \rightarrow \alpha$.

let $compose\ f\ g = \text{fun } x \rightarrow f\ (g\ x);;$

val $compose : (\alpha \rightarrow \beta) \rightarrow (\gamma \rightarrow \alpha) \rightarrow \gamma \rightarrow \beta = \langle \text{fun} \rangle$

let **rec** $power\ f\ n =$

if $n \leq 0$ **then** id **else** $compose\ f\ (power\ f\ (n-1));;$

val $power : (\alpha \rightarrow \alpha) \rightarrow int \rightarrow (\alpha \rightarrow \alpha) = \langle \text{fun} \rangle$

Polymorphism inference

let compose f g x = f (g x);;

val compose : $(\alpha \rightarrow \beta) \rightarrow (\gamma \rightarrow \alpha) \rightarrow \gamma \rightarrow \beta = \langle \text{fun} \rangle$

The type of the function compose is inferred as follows:

- the first argument f is an arbitrary function, hence of type $\alpha \rightarrow \beta$;
- the second argument g is a function whose result is the argument of f, therefore of type α ;
- the domain of g is arbitrary, hence g has type $\gamma \rightarrow \alpha$;
- the function compose has an argument x which is passed to g, hence has type γ ;
- finally, the result of compose is returned by f, hence of type β .

Polymorphic equality

- The **equality operator** in OCaml is polymorphic and built in:
(=);;
- : $\alpha \rightarrow \alpha \rightarrow \text{bool} = \langle \text{fun} \rangle$
- We must be very careful that it coincides with our intended notion of equality, because the compiler will always accept it on **non-functional values**.
- It consists in the **syntactical equality**: two values are equal if they have the same structure and if their respective parts are equal, recursively. It fails on closures.

```
# 1 = 1 && [1;2] = [1;2] && "yes" = "yes";;
```

```
- : bool = true
```

```
# (fun x → x) = (fun x → x);;
```

```
Exception: Invalid_argument "equal: functional value".
```